CONSOLE VIRTUELLE POUR MiSTer · DE10-NANO

MONOCORE-1

MANUEL DU PROGRAMMEUR



Écrivez vos propres jeux · Tutoriels pas à pas · Référence complète

ÉDITION 3 · OCTOBRE 2026

Sommaire

1 La MONOCORE-1 en un coup d'oeil 3
2 Installer l'atelier 5
3 Premier programme : BONJOUR 8
4 Petit cours de C 10
5 La manette 13
6 Les sprites 15
7 Le son 18
8 Le décor 21
9 Projet : ATTRAPE-ÉTOILES 23
10 Sous le capot 27
11 Passer au SDK mc 29
12 Projet : NEBULA 34
13 La haute résolution 45
A La bibliothèque easy 51
B Carte mémoire et registres 53
C Limites et dépannage 55
D Le SDK mc 56

Avant-propos

Bienvenue ! Vous avez entre les mains le manuel de la **MONOCORE-1**, une console de jeu qui n'a jamais existé dans le commerce : elle vit dans le circuit programmable (FPGA) de votre MiSTer. Elle a été conçue pour accueillir des adaptations de jeux d'arcade des années 1978 à 1995, mais aussi, et surtout, **vos propres jeux**.

Comme les ordinateurs familiaux de notre enfance, elle s'allume et attend qu'on lui dise quoi faire. Ce manuel vous apprend à le lui dire. **Aucune connaissance préalable n'est nécessaire** : nous partirons d'un écran vide pour arriver, au chapitre 9, à un jeu complet avec un titre, un score, des vies, de la musique et des explosions.

Chaque exemple de ce manuel est un vrai programme, fourni dans le dossier `manual/examples`. Les images d'écran ont été prises en le faisant tourner : ce que vous voyez est ce que vous obtiendrez.

Une langue pour le code

Comme dans presque tous les langages de programmation, les noms utilisés dans le code (fonctions, variables, fichiers, réglages) sont en **anglais** : `print` pour écrire, `wait_frame` pour attendre l'image suivante... Chaque nom est expliqué la première fois qu'il apparaît, et l'annexe A les traduit tous. Les textes affichés à l'écran, eux, sont en français.

Comment lire ce manuel

- Les chapitres 1 à 3 installent l'atelier et font tourner un premier programme. Lisez-les dans l'ordre.
- Les chapitres 4 à 8 présentent une notion chacun : le langage C, la manette, les sprites, le son, le décor.
- Le chapitre 9 assemble tout dans un jeu. Le chapitre 10 ouvre le capot pour les curieux.
- Les chapitres 11 et 12 présentent le SDK bas niveau mc, pour les jeux plus ambitieux, avec un shoot'em up complet.
- Les annexes servent de référence : gardez-les sous la main pendant que vous programmez.

Bon courage, et amusez-vous bien !

1

La MONOCORE-1 en un coup d'oeil

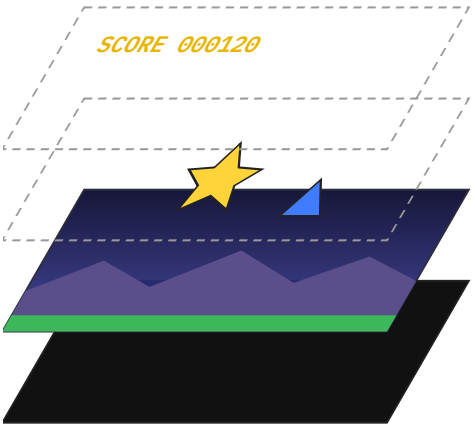
Ce qu'il y a dans la boîte, et comment l'image et le son sont fabriqués.

La MONOCORE-1 est une console complète : un processeur, une mémoire, un circuit vidéo, un circuit sonore, et jusqu'à quatre manettes et un clavier. Un programme de jeu, une **ROM**, est un fichier `.mnc` que l'on charge depuis le menu du MiSTer ou que l'on glisse dans le simulateur.

Élément	Caractéristiques
Processeur	RISC-V 32 bits à 50 MHz, environ 47 millions d'instructions par seconde
Mémoire	128 Ko de mémoire de travail, 32 Ko de mémoire sauvegardée, ROM jusqu'à 32 Mo
Image	320 x 240 pixels par défaut (de 224 x 192 à 384 x 288), 60 images par seconde, 4096 couleurs à l'écran parmi 32 768
Plans	4 plans de tuiles qui défilent, chacun dans sa direction
Sprites	512 objets mobiles, agrandis ou réduits à volonté, retournés, jusqu'à 64 par ligne
Son	8 voix d'échantillons (dont le format ADPCM des bornes d'arcade), 8 générateurs de notes et de bruit, stéréo 48 kHz
Commandes	4 manettes : 4 directions, 6 boutons, START et COIN (pièce), sticks analogiques, paddles et spinners ; un clavier USB
Cartouche	sur la MiSTer, une ou deux mémoires Flash de 8 Mo branchées sur le port USER

Comment se forme l'image

L'écran est construit 60 fois par seconde, ligne par ligne, en superposant plusieurs couches. Avec la bibliothèque **easy** (« facile ») que nous utiliserons dans ce manuel, il y en a quatre :



4. le texte (plan 0)
toujours devant

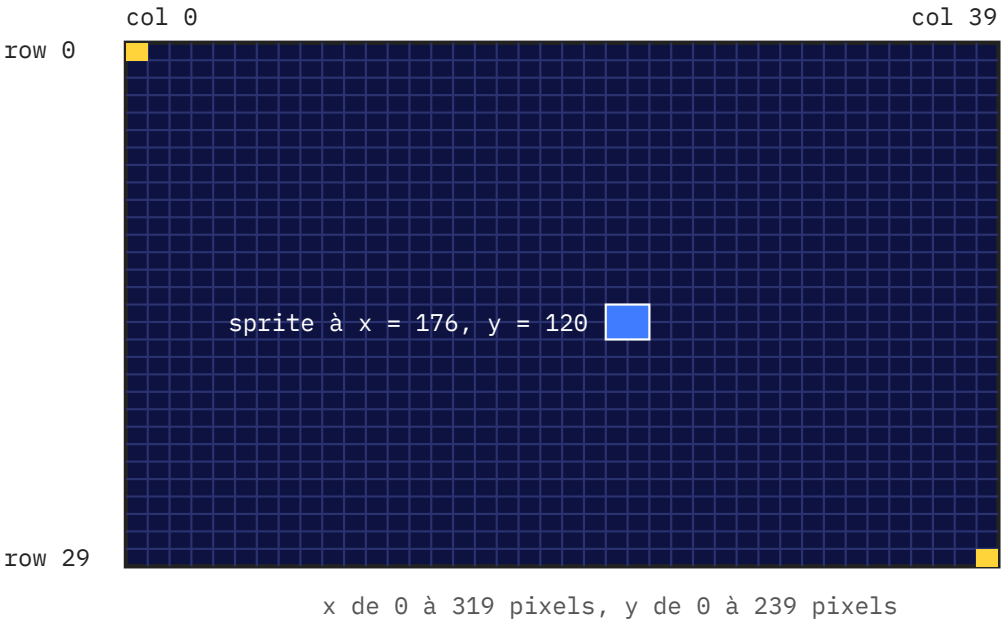
3. les sprites
64 objets mobiles de 16 x 16 à 64 x 64

2. le décor (plan 1)
tuiles de 16 x 16, il défile

1. la couleur de fond
là où rien n'est dessiné

L'image est un empilement : ce qui est dessiné au-dessus cache ce qui est dessous.

Pour placer les choses à l'écran, on utilise deux façons de compter. Le texte se range dans une grille de cases de 8 x 8 pixels : 40 colonnes et 30 lignes. Les sprites, eux, se placent au pixel près : x va de 0 (à gauche) à 319 (à droite), y de 0 (en haut) à 239 (en bas). Attention, y grandit vers le **bas**, comme sur toutes les consoles.



Le texte se place en colonnes et lignes (une case = 8 x 8 pixels), les sprites au pixel près.

Le saviez-vous ?

Une image de la MONOCORE-1 compte 264 lignes, dont 240 visibles. Pendant les 24 lignes invisibles, la **pause verticale**, rien n'est dessiné : c'est le bon moment pour préparer l'image suivante. La fonction `wait_frame()` (« attendre l'image ») attend précisément ce moment.

2

Installer l'atelier

Trois outils gratuits, un simulateur, et la console est prête à recevoir vos programmes.

Un programme MONOCORE-1 s'écrit en **langage C** dans un simple éditeur de texte, puis il est transformé en ROM par l'outil `build.py` (« construire »). Il faut pour cela :

1. **LLVM** (le compilateur `clang` et ses amis) : il sait produire du code pour le processeur RISC-V de la console ;
2. **Python 3** avec la bibliothèque **Pillow**, pour convertir vos dessins et vos sons ;
3. un **éditeur de texte** : n'importe lequel convient (Bloc-notes, TextEdit, Geany, VS Code…).

SOUS LINUX (DEBIAN, UBUNTU, RASPBERRY PI OS…)

```
sudo apt install clang lld llvm python3 python3-pil
```

SOUS MACOS

```
brew install llvm lld python
pip3 install pillow
export PATH="$(brew --prefix llvm)/bin:$PATH"
```

La dernière ligne ajoute les outils LLVM au chemin de recherche des commandes ; ajoutez-la à votre fichier `~/.zshrc` pour ne pas avoir à la retaper.

SOUS WINDOWS

Le plus simple est d'installer **WSL** (le Linux intégré à Windows : dans une invite de commandes, tapez `wsl --install`, redémarrez) puis de suivre les instructions Linux ci-dessus. On peut aussi installer LLVM (llvm.org) et Python (python.org) directement, puis `pip install pillow`.

Dans WSL, les disques de Windows se trouvent sous `/mnt` : `D:` devient `/mnt/d`, et les chemins s'écrivent avec des `/`. Ne recopiez pas un chemin Windows tel quel : bash prend les `\` pour des caractères spéciaux et les supprime. Pour un programme rangé dans `D:\Mes_Documents\Jeux\01_myhello` :

```
cd /mnt/d/Mes_Documents/Jeux/01_myhello
```

Si ce programme n'est pas rangé dans le dossier de la console (voir ci-dessous), appelez `build.py` par son chemin complet. Avec le dossier de la console dans `~/monocore`, comme ci-dessous, la commande ne crée rien d'autre que la ROM :

```
python3 ~/monocore/sdk/build.py
```

La ROM est créée dans le sous-dossier `build` du programme, visible directement depuis Windows. La commande `wslpath 'D:\Mes_Documents'` convertit un chemin Windows en chemin WSL (gardez les apostrophes), et la touche **Tab** complète les noms de dossiers pendant la frappe.

VÉRIFIER QUE TOUT EST PRÊT

TERMINAL deux lignes doivent s'afficher

```
clang -print-targets | grep riscv32
python3 -c "import PIL; print('Pillow OK')"
```

Le dossier de la console

Le projet MONOCORE-1 est un dossier, que l'on récupère une fois pour toutes avec **git** (`sudo apt install git` s'il manque). La commande suivante crée le dossier `monocore` dans votre dossier personnel, que bash note `~` :

```
cd ~
git clone https://github.com/jadiema/monocore.git
```

Plus tard, `git pull` lancé dans ce dossier le met à jour. Dans la suite du manuel, le **dossier de la console** est ce dossier `~/monocore`. Il contient, entre autres :

Dossier	Contenu
`sdk/`	le kit de développement : `easy.h`, `monocore.h`, l'outil `build.py`, la police de caractères
`manual/examples/`	tous les programmes de ce manuel, prêts à construire
`dist/monocore.html`	le simulateur : une page web qui fait tourner les ROM sur votre ordinateur
`mister/`	le core pour le MiSTer

Le simulateur

Ouvrez `dist/monocore.html` dans votre navigateur (Firefox, Chrome, Edge ou Safari). Cliquez sur **Mettre sous tension** : la ROM de test démarre. Pour essayer vos programmes, cliquez sur **Charger une ROM…** ou glissez un fichier `.mnc` sur l'écran. Le simulateur se joue au clavier ou avec une manette USB :

Commande	Joueur 1	Joueur 2	Joueur 3	Joueur 4
Directions	flèches	J I K L	E R F G	pavé 4 8 5 6
Boutons 1 à 3	Z X C	N M ,	Q W V	pavé 0 . Entrée
Boutons 4 à 6	A S D	H U O	—	—
START	1 ou Entrée	2	3	4 ou pavé +
COIN (pièce)	5	6	—	—

Les directions se lisent gauche, haut, bas, droite. Les lettres sont celles marquées sur votre clavier, qu'il soit QWERTY ou AZERTY. Jusqu'à quatre manettes USB ou Bluetooth sont reconnues, une par joueur, sticks analogiques compris. La case **clavier de la console** (touche F9) envoie le clavier au jeu au lieu des joueurs, et le bouton **16:9** étire l'image comme une MiSTer en plein écran sur un écran large.

Sur le côté, le **moniteur** montre la charge du processeur, le nombre de sprites par ligne, les voix sonores actives, et une **console de debug** qui affiche ce que votre programme écrit dans `DEBUG_OUT`, bien pratique pour chercher une erreur.

La console MiSTer

Pour jouer sur la vraie console, copiez le core `MONOCORE.rbf` dans le dossier `_Console` de la carte SD du MiSTer, et vos ROM dans `games/MONOCORE`. Dans le menu du core (touche F12), choisissez **Charger une ROM**. Une ROM nommée `boot.rom` dans ce dossier se lance toute seule : c'est là que l'on place l'écran d'accueil de la console, AUBE (`examples/aube`).

Une ROM peut aussi venir d'une **cartouche** : une ou deux mémoires Flash SPI W25Q64 (16 Mo au total) branchées sur le port USER du MiSTer, qui contiennent le fichier `.mnc` tel quel. L'outil `tools/cart/cart_split.py` prépare les images à graver, et `mister/LISEZMOI.md` donne le câblage et les réglages du menu. Pour le jeu, rien ne change : la ROM est copiée dans la SDRAM comme si elle venait de la carte SD.

> **Attention**
>
> Le MiSTer doit être équipé d'une **barrette SDRAM** : la ROM du jeu est chargée dedans.

3

Premier programme : BONJOUR

Six lignes de C, une commande, et la console vous salue.

Tout programme vit dans son propre dossier. Celui-ci s'appelle `01_hello` et contient deux fichiers. Le premier, `project.txt`, décrit le projet : le nom du fichier à produire (`name`), le titre inscrit dans la ROM (`title`) et les fichiers du programme (`sources`).

LISTING project.txt

```
name    = hello
title   = BONJOUR
sources = main.c
```

Le second, `main.c`, est le programme lui-même :

LISTING main.c

```
1 #include "easy.h"
2
3 int main(void) {
4   init();
5   background(0, 0, 120);
6   print(14, 12, "BONJOUR !");
7   print(8, 16, "JE SUIS LA MONOCORE-1.");
8   for (;;) wait_frame();
9 }
```

Construire la ROM

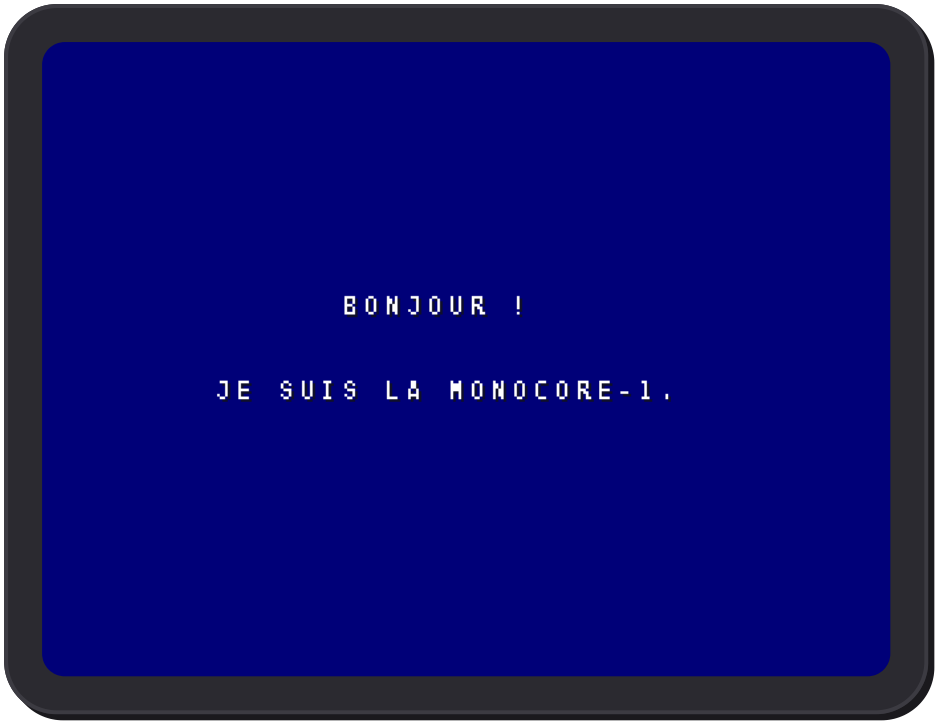
Ouvrez un terminal, placez-vous dans le dossier du programme, et lancez `build.py` en indiquant où il se trouve :

```
cd ~/monocore/manual/examples/01_hello
python3 ../../../sdk/build.py

ROM ready: /home/.../monocore/manual/examples/01_hello/build/hello.mnc
```

`../../../` remonte de trois dossiers, de `01_hello` à `monocore` : `../../../sdk/build.py` est donc `~/monocore/sdk/build.py`, et l'on peut écrire l'un ou l'autre. Le chemin complet a l'avantage de marcher depuis n'importe quel dossier, y compris pour vos propres programmes rangés ailleurs. `build.py` crée le sous-dossier `build` dans le dossier du programme et y range la ROM.

Le fichier `build/hello.mnc` est votre ROM. Chargez-la dans le simulateur :



BONJOUR en 320 x 240 : le texte blanc sur le fond bleu.

Ligne par ligne

Ligne	Ce qu'elle fait
1	`#include "easy.h"` donne accès à la bibliothèque easy : toutes les fonctions de ce manuel.
3	`int main(void)` : c'est ici que le programme commence. Tout ce qui est entre les accolades `{ }` s'exécute dans l'ordre.
4	`init()` (« initialiser ») prépare la console : police, couleurs, écran allumé. À appeler une fois, en premier.
5	`background(0, 0, 120)` choisit la couleur du fond : rouge 0, vert 0, bleu 120 (sur 255).
6 et 7	`print(colonne, ligne, "texte")` (« imprimer ») écrit un texte dans la grille de 40 x 30 cases.
8	`for (;;)` répète sans fin ce qui suit. Un programme de console ne s'arrête jamais : ici, il attend image après image avec `wait_frame()`.

À vous de jouer

- Changez le texte, sa position, la couleur du fond, puis reconstruisez la ROM.
- Les accents du français s'affichent (é, è, à, ç, « »…) ; les autres lettres accentuées perdent leur accent.
- Une erreur ? `build.py` indique le fichier et la ligne. Le plus souvent, il manque un point-virgule `;` en fin de ligne.

4

Petit cours de C

Variables, conditions et boucles : de quoi donner vie à l'écran.

Le C est le langage dans lequel ont été écrits une grande partie des jeux et des systèmes depuis les années 80. Nous n'en utiliserons qu'une petite partie, présentée ici sur un exemple qui affiche les huit encres de texte, compte les secondes et fait clignoter un message.

LISTING 02_colors/main.c

```c
#include "easy.h"

int main(void) {
  init();
  background(20, 0, 60);                    // a very dark purple

  print_ink(12, 3, "LES HUIT ENCRES", 1);    // ink 1: yellow
  for (int n = 0; n < 8; n++) {              // n takes the values 0, 1, ... up to 7
    print_ink(12, 7 + n * 2, "ENCRE NUMERO", n);
    print_num(25, 7 + n * 2, n, 1, n);
  }

  int seconds = 0;
  for (;;) {                                 // forever
    wait_frame();
    if (frame_count % 60 == 0) seconds = seconds + 1;   // 60 frames = 1 second
    print(9, 25, "SECONDES :");
    print_num(20, 25, seconds, 4, 2);

    // blinking text: on for half a second, off for half a second
    if ((frame_count / 30) % 2 == 0) print_ink(12, 27, "APPUIE SUR START", 6);
    else                             print    (12, 27, "                ");

    if (pressed(1, PAD_START)) background(random(256), random(256), random(256));
  }
}
```



Les huit encres, le compteur de secondes et le message qui clignote.

Les variables

Une **variable** est une case de mémoire qui porte un nom et contient un nombre. `int seconds = 0;` crée la variable `seconds` (« secondes ») et y range 0. Ensuite, `seconds = seconds + 1;` lit la valeur, ajoute 1 et range le résultat. `int` veut dire « nombre entier » (de -2 milliards à +2 milliards).

La bibliothèque fournit une variable toute prête : `frame_count` (« nombre d'images »), qui compte les images depuis le démarrage. Comme il y en a 60 par seconde, `frame_count % 60 == 0` (le reste de la division par 60 vaut zéro) est vrai une fois par seconde.

Les conditions

`if (condition) action;` n'exécute l'action que si la condition est vraie. On peut ajouter `else autre_action;` pour le cas contraire. Les comparaisons s'écrivent `==` (égal, avec deux signes !), `!=` (différent), `<`, `>`, `<=`, `>=`. On combine avec `&&` (et) et `||` (ou).

Les boucles

`for (int n = 0; n < 8; n++)` répète ce qui suit en faisant prendre à `n` les valeurs 0, 1, 2... jusqu'à 7. `n++` veut dire « ajouter 1 à n ». La forme `for (;;)`, sans rien, répète pour toujours : c'est la **boucle principale** de tout jeu, qui tourne une fois par image.

Les fonctions

`print_ink(...)`, `background(...)` ou `random(256)` sont des **fonctions** : des morceaux de programme qui ont un nom, reçoivent des valeurs entre parenthèses et parfois en rendent une. `random(256)` (« au hasard ») rend un nombre entre 0 et 255. Vous écrirez vos propres fonctions au chapitre 9.

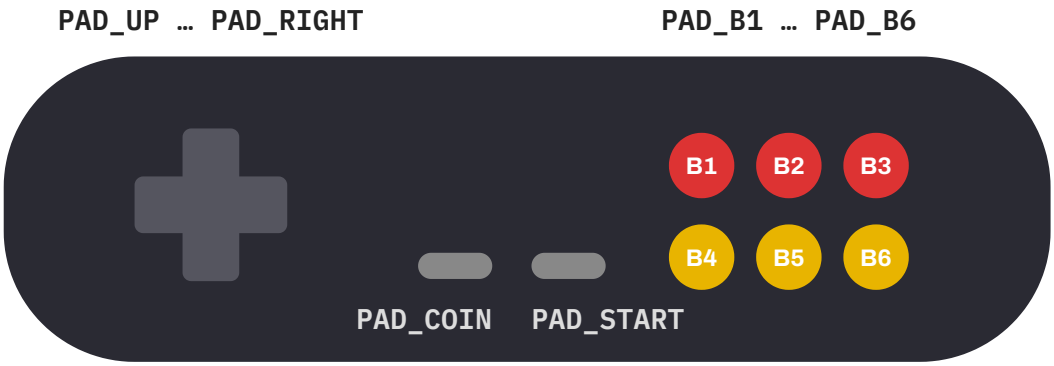
La recette de tout jeu

```
int main(void) {
  init();
  // 1. set up: colours, pictures, map, variables
  for (;;) {
    wait_frame();
    // 2. read the joypad
    // 3. move the world, check collisions
    // 4. draw: sprites, score
  }
}
```

Préparer, puis pour chaque image : lire la manette, faire bouger le monde, afficher.

La manette

Savoir ce que fait le joueur, image après image.



Les noms des commandes, tels qu'on les écrit dans un programme : tous commencent par `PAD_` *(pad : manette).*

Deux fonctions suffisent pour tout savoir des manettes :

Fonction	Rend vrai quand…	Pour…
`held(1, PAD_LEFT)`	le bouton est enfoncé en ce moment (*held* : tenu)	les déplacements, le tir automatique
`pressed(1, PAD_B1)`	le bouton vient d'être enfoncé, une seule fois par appui (*pressed* : appuyé)	sauter, valider un menu, compter

Le premier nombre est le joueur, de 1 à 4. Les manettes sont lues une fois par image, par `wait_frame()`. Voici un programme d'essai qui affiche l'état de chaque commande des quatre manettes :

LISTING 03_joypad/main.c

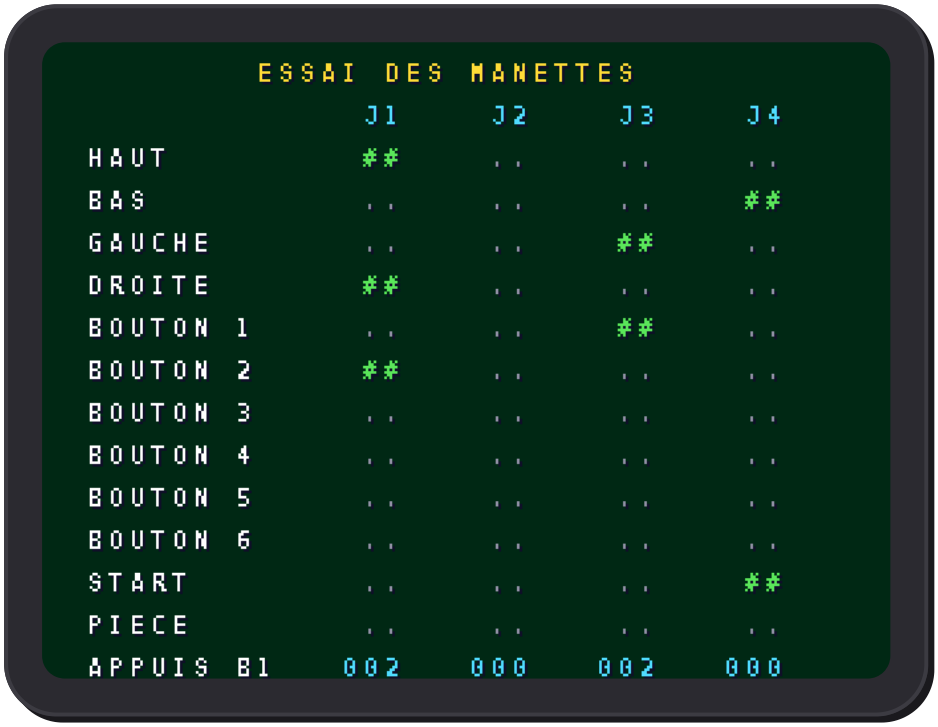
```
#include "easy.h"

// a label and a button for each line of the display
static const char *labels[12] = {
  "HAUT", "BAS", "GAUCHE", "DROITE", "BOUTON 1", "BOUTON 2",
  "BOUTON 3", "BOUTON 4", "BOUTON 5", "BOUTON 6", "START", "PIECE"};
static const uint32_t buttons[12] = {
  PAD_UP, PAD_DOWN, PAD_LEFT, PAD_RIGHT, PAD_B1, PAD_B2,
  PAD_B3, PAD_B4, PAD_B5, PAD_B6, PAD_START, PAD_COIN};

int main(void) {
  init();
  background(0, 40, 30);
  print_ink(10, 1, "ESSAI DES MANETTES", 1);
  int presses[5] = {0, 0, 0, 0, 0};       // presses[1] to presses[4]: one for each
player

```

```
  for (int player = 1; player <= 4; player++) {
    print_ink(9 + player * 6, 3, "J", 2);
    print_num(10 + player * 6, 3, player, 1, 2);
  }
  for (;;) {
    wait_frame();
    for (int i = 0; i < 12; i++) {
      print(2, 5 + i * 2, labels[i]);
      for (int player = 1; player <= 4; player++) {
        if (held(player, buttons[i])) print_ink(9 + player * 6, 5 + i * 2, "##", 3);
        else                          print_ink(9 + player * 6, 5 + i * 2, "..", 7);
      }
    }
    print(2, 29, "APPUIS B1");
    for (int player = 1; player <= 4; player++) {
      if (pressed(player, PAD_B1)) presses[player]++;   // once per press, even if the button stays down
      print_num(8 + player * 6, 29, presses[player], 3, 2);
    }
  }
}
```



Le joueur 1 tient HAUT, DROITE et le bouton 2, le joueur 3 GAUCHE et le bouton 1, le joueur 4 BAS et START.

Ce programme utilise deux **tableaux** : `labels[12]` contient douze textes, `buttons[12]` douze boutons. `labels[0]` est le premier, `labels[11]` le dernier. La boucle `for` les parcourt tous avec la même variable `i` ; une seconde boucle, à l'intérieur, passe en revue les joueurs. Le tableau `presses[5]` garde un compteur par joueur : on lui a donné une case de plus pour écrire `presses[1]` à `presses[4]`, la case 0 reste inutilisée.

Joueurs 3 et 4

Les manettes 3 et 4 existent depuis la version 2 de la console. Sur un core MiSTer plus ancien, `held(3, …)` et `held(4, …)` répondent toujours faux : le jeu reste jouable à un ou deux.

6

Les sprites

Des objets qui bougent : votre héros, les ennemis, les bonus.

Dessiner une planche

Les dessins des sprites se rangent dans une **planche** (*sprite sheet*) : une image PNG de **16 pixels de large**, où les dessins de 16 x 16 sont posés les uns sous les autres. Le premier porte le numéro 0, le suivant le numéro 1, et ainsi de suite. Dessinez-la avec n'importe quel logiciel de dessin (Aseprite, LibreSprite, GIMP, Paint...) en respectant trois règles :

- une **seule colonne** de dessins, de haut en bas : la largeur de l'image est celle d'un dessin ;
- **15 couleurs au plus** pour toute la planche ;
- le **transparent** (les pixels effacés de l'image) laisse voir ce qui est derrière le sprite.



N°	Nom dans le code	Dessin
0	`SHIP`	vaisseau
1	`STAR`	étoile
2	`STAR2`	étoile qui scintille
3	`BOMB`	bombe
4	`EXPLOSION`	explosion
5	`HEART`	cœur

La planche des exemples, agrandie huit fois. Elle est dans `04_sprites/sheet.png`.

Il suffit de la nommer dans `project.txt` (`sprites = sheet.png`) : `build.py` la convertit, et `init()` la charge avec ses couleurs. Dans le programme, `sheet` (le nom du fichier sans `.png`) est le numéro de son premier dessin : `sheet + 2` est son troisième dessin. Ce nom est prêt à l'emploi, il n'y a rien à déclarer.

Un jeu peut avoir plusieurs planches : `sprites = hero.png enemies.png`. Leurs dessins se numérotent à la suite, et `hero + 1` ou `enemies + 3` les désignent sans avoir à compter. Chaque planche garde ses propres 15 couleurs.

De plus grands sprites

Pour un héros plus détaillé, une planche peut faire **32 ou 64 pixels de large**, avec des dessins de 32 x 32 ou de 64 x 64 posés les uns sous les autres. Chaque dessin garde la taille de sa planche : un héros de 64 x 64 et des bonus de 16 x 16 se côtoient sans rien changer au programme.

Afficher et déplacer

LISTING 04_sprites/main.c

```c
#include "easy.h"

// pictures of sheet.png: sheet is the number of its first picture
#define SHIP   (sheet + 0)
#define STAR   (sheet + 1)
#define STAR2  (sheet + 2)

int main(void) {
  init();                          // also loads sheet.png
  background(0, 0, 40);
  print_ink(3, 1, "MANETTE:BOUGER  B1:MIROIR  B2:TAILLE", 2);

  int x = 152, y = 120;            // ship position
  int flipped = 0, big = 0;

  for (;;) {
    wait_frame();
    if (held(1, PAD_LEFT))  x = x - 2;
    if (held(1, PAD_RIGHT)) x = x + 2;
    if (held(1, PAD_UP))    y = y - 2;
    if (held(1, PAD_DOWN))  y = y + 2;
    if (pressed(1, PAD_B1)) { flipped = !flipped; flip_sprite(0, 0, flipped); }
    if (pressed(1, PAD_B2)) { big = !big;         scale_sprite(0, big ? 200 : 100); }
    sprite(0, x, y, SHIP);

    // five twinkling stars: the picture changes every 10 frames
    for (int i = 0; i < 5; i++) {
      int picture = ((frame_count / 10 + i) % 2) ? STAR : STAR2;
      sprite(1 + i, 30 + i * 60, 50, picture);
    }
  }
}
```



Le vaisseau agrandi deux fois (bouton 2), et les étoiles qui scintillent.

Fonction	Effet
`sprite(n, x, y, picture)`	affiche le sprite n (0 à 63) à la position x, y avec le dessin choisi
`hide_sprite(n)`	fait disparaître le sprite n
`flip_sprite(n, h, v)`	le retourne horizontalement (h = 1) et/ou verticalement (v = 1)
`scale_sprite(n, 200)`	l'agrandit (200 = double, 50 = moitié)
`collide(a, b)`	vrai si les sprites a et b se touchent

Animer

Pour animer un sprite, on change de dessin régulièrement. `(frame_count / 10) % 2` vaut 0 pendant 10 images, puis 1 pendant 10 images, etc. : de quoi alterner deux dessins six fois par seconde. Avec `% 4`, on fait tourner quatre dessins.

Les limites

On peut afficher jusqu'à **64 sprites sur une même ligne** de l'écran, et 1280 pixels de sprites en tout sur cette ligne : c'est 80 sprites de 16 x 16 mais seulement **20 sprites de 64 x 64** côte à côte. Au-delà, les derniers ne sont pas dessinés sur cette ligne : le moniteur du simulateur l'indique en rouge. Un jeu a au plus **8 planches et décors** en tout, car chacun prend l'une des 8 palettes de couleurs.

7

Le son

Des bips, des bruits, des mélodies et des échantillons.

La MONOCORE-1 a **8 canaux** de notes (numérotés de 0 à 7), qui jouent ensemble, et **8 voix d'échantillons** pour les sons enregistrés.

Fonction	Effet
`tone(channel, NOTE_A4, 150)`	joue un son carré de 440 hertz (le LA du diapason) au volume 150 sur 255
`noise(channel, 120)`	un souffle : explosions, vagues, caisse claire
`quiet(channel)`	fait taire le canal
`play(channel, tune, 15, 100)`	joue une mélodie en boucle, toute seule
`stop(channel)`	arrête la mélodie
`play_sample(voice, sound, volume)`	joue un son enregistré (fichier WAV)

La fréquence de `tone()` est en hertz, mais il est plus simple d'écrire le nom de la note : `easy.h` donne un nom `NOTE_` à chaque note, des octaves 2 à 7. Le nom suit la notation anglaise (voir plus bas), avec un **S** (*sharp*) pour le dièse ou un **b** pour le bémol, puis le numéro de l'octave. Comme sur un piano, DO dièse et RÉ bémol sont la même touche, donc la même fréquence : la console joue en **tempérament égal**, où l'octave est partagée en 12 demi-tons identiques (dans les accords anciens, les deux notes diffèrent un peu).

Nom	Note	Hertz
`NOTE_C4`	DO (le DO du milieu du clavier)	262
`NOTE_CS4`	DO dièse	277
`NOTE_Db4`	RÉ bémol (la même note)	277
`NOTE_A4`	LA du diapason	440
`NOTE_Bb4`	SI bémol	466
`NOTE_B4`	SI	494
`NOTE_C5`	DO de l'octave au-dessus	523

LISTING 05_sound/main.c

```
1 #include "easy.h"
2
3 // "Au clair de la lune", one note per beat
4 static const char *moonlight =
```

```
  "C C C D E - D - C E D D C - - . "
  "C C C D E - D - C E D D C - - . ";

int main(void) {
  init();
  background(40, 0, 0);
  print_ink(13, 3, "LE PETIT PIANO", 1);
  print(4, 8,  "BOUTONS 1 A 6 : DO RE MI FA SOL LA");
  print(4, 10, "HAUT          : SI");
  print(4, 12, "BAS           : BRUIT");
  print(4, 14, "START         : MUSIQUE OUI / NON");

  static const int notes[6] = {NOTE_C4, NOTE_D4, NOTE_E4, NOTE_F4, NOTE_G4, NOTE_A4};
  static const uint32_t keys[6] = {PAD_B1, PAD_B2, PAD_B3, PAD_B4, PAD_B5, PAD_B6};
  int playing = 0;

  for (;;) {
    wait_frame();
    int frequency = 0;
    for (int i = 0; i < 6; i++)
      if (held(1, keys[i])) frequency = notes[i];
    if (held(1, PAD_UP)) frequency = NOTE_B4;
    if (frequency) tone(0, frequency, 150); else quiet(0);
    if (held(1, PAD_DOWN)) noise(1, 120); else quiet(1);

    if (pressed(1, PAD_START)) {
      playing = !playing;
      if (playing) play(2, moonlight, 15, 100);
      else         stop(2);
    }
    print_ink(12, 20, playing ? "MUSIQUE : OUI" : "MUSIQUE : NON", playing ? 3 : 7);
  }
}
```



Le petit piano : chaque bouton joue une note.

Écrire une mélodie

Une mélodie est un texte où chaque mot est un **temps**. Les notes s'écrivent avec les lettres de la notation anglaise :

DO	Ré	MI	FA	SOL	LA	SI
C	D	E	F	G	A	B

écriture	Signification
`C D E F G A B`	les notes, dans l'octave 4 (celle du LA à 440 Hz)
`C5 G3`	un chiffre après la note change d'octave : C5 est le DO aigu suivant
`F# C#5`	un # (dièse) monte la note d'un demi-ton
`Bb Eb5`	un b (bémol) la descend d'un demi-ton
`-`	prolonge la note précédente d'un temps
`.`	un temps de silence

Le troisième nombre de `play()` règle le tempo : le nombre d'images par temps. 15 images, c'est 4 temps par seconde (240 à la minute) ; 30, deux fois plus lent. Pour jouer plusieurs voix en même temps (une mélodie et une basse), lancez deux mélodies sur deux canaux.

Les sons enregistrés

Un fichier WAV nommé dans `project.txt` (`sounds = boom.wav`) devient le nom `boom`. On le joue sur la voix 0, au volume 230, avec :

```
play_sample(0, boom, 230);
```

Bonne pratique

Gardez des sons courts et en 8 000 à 22 050 Hz : ils occupent moins de place dans la ROM et sonnent très « arcade ». Un son de 16 000 Hz pendant une demi-seconde occupe 8 Ko.

Le décor

Un grand paysage en tuiles, qui défile derrière les sprites.

Le décor (*map*) est une grande image découpée en **tuiles de 16 x 16 pixels**. Les tuiles identiques ne sont stockées qu'une fois : un ciel uni ou un sol répétitif ne coûtent presque rien. Le décor peut mesurer jusqu'à **1024 x 1024 pixels** et il boucle : en défilant au-delà du bord droit, on retrouve le bord gauche.

- Dessinez une image dont la largeur et la hauteur sont des multiples de 16, en **15 couleurs** au plus ;
- nommez-la dans `project.txt` : `maps = landscape.png` ;
- `init()` l'affiche, avec ses couleurs ; s'il y a plusieurs décors (`maps = level1.png level2.png`), c'est le premier, et `load_map(level2)` passe au suivant.

LISTING 06_scroll/main.c

```c
#include "easy.h"

int main(void) {
  init();                    // loads sheet.png and shows the map landscape.png
  print_ink(2, 1, "GAUCHE / DROITE : VITESSE", 1);

  int camera = 0;            // screen position in the map, in pixels
  int speed = 1;
  int y = 100;

  for (;;) {
    wait_frame();
    if (pressed(1, PAD_RIGHT) && speed < 6)  speed++;
    if (pressed(1, PAD_LEFT)  && speed > -6) speed--;
    if (held(1, PAD_UP)   && y > 20)  y -= 2;
    if (held(1, PAD_DOWN) && y < 160) y += 2;

    camera = camera + speed;
    scroll_map(camera, 0);               // the map is 1024 pixels wide and wraps around by itself

    sprite(0, 152, y, 0);
    print(2, 28, "VITESSE :");
    print_num(12, 28, speed, 2, 2);
  }
}
```



Le paysage de 1024 pixels défile derrière le vaisseau.

`scroll_map(x, y)` (« faire défiler le décor ») indique quel point du décor apparaît dans le coin en haut à gauche de l'écran. En augmentant `x` à chaque image, le décor glisse vers la gauche : on a l'impression d'avancer vers la droite.

Le saviez-vous ?

Les jeux des années 80 donnaient de la profondeur en faisant défiler plusieurs plans à des vitesses différentes : le lointain lentement, le premier plan vite. La MONOCORE-1 a quatre plans, et peut même faire défiler chaque ligne de l'écran différemment (chapitre 10).

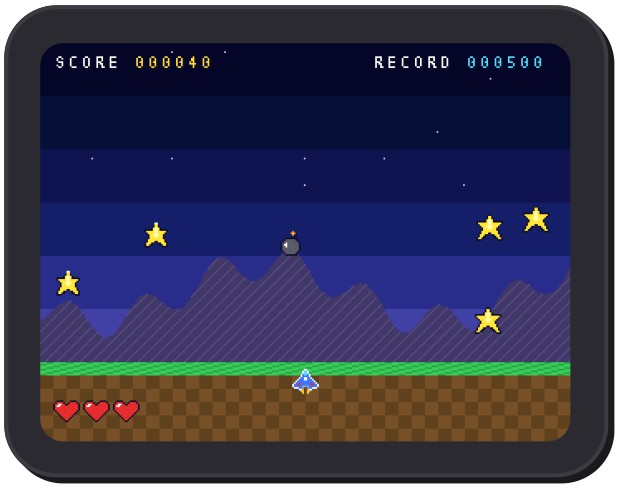
9

Projet : ATTRAPE-ÉTOILES

Un jeu complet : écran titre, score, record, vies, musique et explosions.



L'écran titre.



Une partie en cours.

Le principe : des étoiles et des bombes tombent du ciel. Déplacez le vaisseau avec `PAD_LEFT` et `PAD_RIGHT` pour attraper les étoiles (10 points chacune) et éviter les bombes (une vie perdue). Plus le score monte, plus tout tombe vite. Le jeu tient en 150 lignes, rangées en **fonctions** qui ont chacune un rôle.

Organisation du programme

Fonction	Rôle
`main`	prépare la console, puis enchaîne pour toujours : titre, partie, fin de partie
`title_screen`	écran titre : affiche le titre et les règles, joue la musique, attend START
`game`	la partie : manette, objets, explosion, score, jusqu'à ce qu'il n'y ait plus de vies
`drop_item`	« lâcher un objet » : le place en haut de l'écran, au hasard, étoile ou bombe
`move_items`	fait tomber les objets et teste s'ils touchent le vaisseau
`show_score`	écrit le score et le record, dessine les cœurs
`game_over`	affiche PARTIE TERMINEE pendant trois secondes

Les structures

Chaque objet qui tombe a une position, une vitesse, et peut être une étoile ou une bombe. Plutôt que d'avoir quatre tableaux séparés, on regroupe ces informations dans une **structure** : `item_t` (« objet »). Le tableau `items[6]` contient six objets, et `it->y` désigne la position verticale de l'objet `it`.

Le listing complet

LISTING 07_stars/project.txt

```
# Attrape-Etoiles: the complete game of chapter 9
name    = stars
title   = ATTRAPE-ETOILES
sources = main.c
sprites = sheet.png
maps    = landscape.png
sounds  = boom.wav
```

LISTING 07_stars/main.c

```
// ATTRAPE-ETOILES - the complete game of the MONOCORE-1 manual
#include "easy.h"

// pictures of sheet.png
#define SHIP      (sheet + 0)
#define STAR      (sheet + 1)
#define STAR2     (sheet + 2)
#define BOMB      (sheet + 3)
#define EXPLOSION (sheet + 4)
#define HEART     (sheet + 5)

// sprites in use
#define SPR_PLAYER  0
#define SPR_ITEM    1      // 1 to 6: the falling items
#define NUM_ITEMS   6
#define SPR_LIFE    10     // 10 to 12: the hearts

typedef struct {
  int active;
  int x, y;              // position in pixels
  int speed;             // pixels per frame
  int bomb;              // 1 = bomb, 0 = star
} item_t;

static item_t items[NUM_ITEMS];
static int player_x, score, best, lives, explosion;

static const char *tune =
  "E G C5 - B A G - A B C5 D5 E5 - - . "
  "E5 D5 C5 - B A G - F A G E D - - . ";

// ------------------------------------------------------------------ display
static void show_score(void) {
  print(1, 1, "SCORE");
  print_num(7, 1, score, 6, 1);
  print(25, 1, "RECORD");
  print_num(32, 1, best, 6, 2);
  for (int i = 0; i < 3; i++) {
    if (i < lives) sprite(SPR_LIFE + i, 8 + i * 18, 214, HEART);
    else hide_sprite(SPR_LIFE + i);
  }
}

static void hide_all(void) {
  for (int n = 0; n < 64; n++) hide_sprite(n);
}

// ------------------------------------------------------------------- items
static void drop_item(item_t *it) {
  it->active = 1;
```

```
  it->x = 8 + random(288);
  it->y = -16 - random(80);
  it->speed = 1 + random(2) + score / 300;      // faster and faster
  if (it->speed > 6) it->speed = 6;
  it->bomb = random(4) == 0;                    // one chance in four
}

static void move_items(void) {
  for (int i = 0; i < NUM_ITEMS; i++) {
    item_t *it = &items[i];
    it->y = it->y + it->speed;
    if (it->y > 240) drop_item(it);             // out at the bottom: drop it again from
the top

    int picture = it->bomb ? BOMB : (((frame_count / 8 + i) % 2) ? STAR : STAR2);
    sprite(SPR_ITEM + i, it->x, it->y, picture);

    if (collide(SPR_ITEM + i, SPR_PLAYER)) {
      if (it->bomb) {
        lives = lives - 1;
        explosion = 30;                         // 30 frames of explosion
        play_sample(0, boom, 230);
      } else {
        score = score + 10;
        tone(1, 1200, 120);                     // a short "ding"
      }
      drop_item(it);                            // back to the top, out of the screen
      hide_sprite(SPR_ITEM + i);                // until it is shown there, at the next
frame
    }
  }
}

// ------------------------------------------------------------------ screens
static void title_screen(void) {
  cls();
  hide_all();
  print_ink(11, 7, "ATTRAPE-ETOILES", 1);
  print(6, 11, "ATTRAPE LES ETOILES : 10 POINTS");
  print(6, 13, "EVITE LES BOMBES    : -1 VIE");
  print_ink(11, 27, "UN JEU MONOCORE-1", 0);
  print(25, 1, "RECORD");
  print_num(32, 1, best, 6, 2);
  play(2, tune, 10, 90);

  for (;;) {
    wait_frame();
    if ((frame_count / 30) % 2 == 0) print_ink(12, 18, "APPUIE SUR START", 6);
    else                             print    (12, 18, "                ");
    for (int i = 0; i < 4; i++)                 // four stars dancing under the rules
      sprite(SPR_ITEM + i, 152 + (i - 2) * 50 + 25, 118 + ((frame_count / 3 + i * 3) %
8), STAR);
    scroll_map(frame_count / 2, 0);
    if (pressed(1, PAD_START)) break;
  }
  stop(2);
}

static void game(void) {
  cls();
  hide_all();
  score = 0; lives = 3; explosion = 0;
  player_x = 152;
  for (int i = 0; i < NUM_ITEMS; i++) drop_item(&items[i]);

  while (lives > 0) {
    wait_frame();
    if (held(1, PAD_LEFT)  && player_x > 4)   player_x -= 3;
    if (held(1, PAD_RIGHT) && player_x < 300) player_x += 3;
    if (frame_count % 8 == 0) quiet(1);         // ends the "ding"

    if (explosion > 0) {
      explosion--;
      sprite(SPR_PLAYER, player_x, 196, EXPLOSION);
    } else {
```

```
      sprite(SPR_PLAYER, player_x, 196, SHIP);
      move_items();                          // the game pauses during the explosion
    }
    if (score > best) best = score;
    show_score();
    scroll_map(frame_count, 0);
  }
}

static void game_over(void) {
  for (int i = 0; i < NUM_ITEMS; i++) hide_sprite(SPR_ITEM + i);
  print_ink(12, 12, "PARTIE TERMINEE", 4);
  for (int t = 0; t < 180; t++) {              // 3 seconds, or START
    wait_frame();
    if (pressed(1, PAD_START)) break;
  }
}

int main(void) {
  init();                                      // loads sheet.png and landscape.png
  background(0, 0, 0);
  best = 500;
  for (;;) {
    title_screen();
    game();
    game_over();
  }
}
```

Pour aller plus loin

Le jeu est à vous : modifiez-le ! Quelques idées, de la plus simple à la plus ambitieuse :

1. Changez les couleurs, la vitesse, le nombre de points par étoile.
2. Ajoutez un objet rare (le cœur, dessin 5) qui rend une vie.
3. Faites défiler le décor plus vite quand le score augmente.
4. Ajoutez une deuxième voix à la musique (une basse sur le canal 3).
5. Mode à plusieurs : jusqu'à quatre vaisseaux, un par manette (`held(2, PAD_LEFT)` pour le joueur 2…).
6. Ajoutez le tir : le bouton 1 lance un projectile qui détruit les bombes.

10

Sous le capot

La bibliothèque easy n'est qu'un raccourci : voici ce que la console sait faire d'autre.

Toutes les fonctions de `easy` écrivent dans des **registres** : des adresses mémoire spéciales qui commandent les circuits de la console. `monocore.h` leur donne des noms. Vous pouvez les utiliser à côté de `easy`, à condition de respecter son rangement (plans 0 et 1, palettes 0 à 15 : les 8 encres puis les 8 palettes de couleurs, sprites 0 à 63) ou de ne pas l'utiliser du tout.

Changer la taille de l'écran

```
SCREEN_W = 256;   // from 224 to 384 pixels wide
SCREEN_H = 224;   // from 192 to 288 lines
```

La nouvelle taille prend effet à l'image suivante. Pour un jeu vertical (comme Pac-Man ou Galaga), on choisit 224 x 288 et on demande que l'image soit tournée d'un quart de tour avec `DISP_CTRL |= DISP_ROT(1)`. La ligne `rotation = 1` de `project.txt` inscrit cette orientation dans la ROM, pour que l'écran soit tourné avant même que le jeu démarre. Le simulateur le fait déjà, le core MiSTer le fera dans une prochaine version.

Les quatre plans

Les plans 2 et 3, inutilisés par `easy`, se règlent comme le décor : `LAYER_CTRL(n)` (taille des tuiles, 4 ou 8 bits par pixel, taille de la carte, priorité), `LAYER_CHR(n)` (adresse des dessins), `LAYER_SCROLLX(n)` et `LAYER_SCROLLY(n)`, et la carte `TILEMAP(n)`. Il faut ensuite les allumer : `DISP_CTRL |= DISP_LAYER(2)`.

Le défilement ligne par ligne

Avec l'option `LC_LINESCROLL`, chaque ligne de l'écran ajoute son propre décalage au défilement du plan : routes de jeux de course, vagues, effets de chaleur.

```
LAYER_CTRL(1) |= LC_LINESCROLL;
for (int y = 0; y < 240; y++) {
  LINESCROLL(1)[y].dx = (y > 120) ? (y - 120) * (frame_count % 64) / 32 : 0;
  LINESCROLL(1)[y].dy = 0;
}
```

Les sprites sans limites

La console gère **512 sprites** (la table `SPRITES[n]`), de 8 x 8 à 256 x 256 pixels (largeur et hauteur en cellules de 8 pixels), en 16 ou 256 couleurs, avec un zoom indépendant en largeur et en hauteur (`zoomx`, `zoomy` : 256 = taille normale). Le registre `SPR_COUNT` dit combien d'entrées la console parcourt.

Les interruptions

Une **interruption** arrête le programme un instant pour exécuter `irq_handler()`. `easy` utilise celle du début de la pause verticale (VBLANK). Il en existe d'autres : LINE (au début d'une ligne choisie par `LINE_CMP`, pour changer un registre en cours d'image), TIMER0 et TIMER1 (à intervalle régulier, en cycles du processeur : `T0_RELOAD = CPU_HZ / 240` pour 240 fois par seconde), DMA (fin de transfert).

Le DMA

Le DMA copie ou remplit la mémoire bien plus vite que le processeur : `dma_copy(destination, source, octets)` et `dma_fill(destination, motif, octets)`. Idéal pour charger une palette, une carte de décor, ou des dessins dans la **CHR RAM** (128 Ko de graphismes modifiables en cours de jeu, à l'adresse `CHRRAM_BASE`).

Le temps du processeur

Le processeur exécute environ **780 000 instructions par image**. C'est énorme : un 68000 de borne d'arcade en exécutait 25 000. Le moniteur du simulateur affiche la charge : si elle dépasse 100 %, le jeu ralentit. Gardez une marge, car le simulateur estime le temps à environ 10 % près.

Pour utiliser tout cela sans écrire les registres un par un, le chapitre 11 présente le SDK **mc**.

Règle d'or

Ne comptez jamais sur un nombre de cycles pour régler la vitesse d'un jeu : synchronisez-vous toujours sur les images (`wait_frame()`) ou sur un timer. C'est ce qui garantit que le jeu tourne pareil dans le simulateur et sur la console.

11

Passer au SDK mc

Quand easy devient trop petit : toute la console, toujours en C, sans assembleur.

La bibliothèque `easy` range la console d'une façon simple et fixe : un plan de texte, un décor, 64 sprites de 16 x 16, 32 x 32 ou 64 x 64. C'est parfait pour apprendre, mais un jeu plus ambitieux, ou l'adaptation d'un jeu existant, a besoin de tout le reste : les quatre plans, les 512 sprites, les interruptions, la mémoire. Le chapitre 10 a montré qu'on peut tout faire avec les registres ; le SDK **mc** (pour *MONOCORE*) donne à ces registres des fonctions pratiques, sans rien cacher.

	easy	mc
Pour qui	débuter, petits jeux	jeux complexes, adaptations
Plans	texte (0) et décor (1)	les quatre, réglés librement
Sprites	64, de 16 x 16, 32 x 32 ou 64 x 64	512, de 8 x 8 à 256 x 256, zoom
Interruptions	cachées	VBLANK, deux timers, effets raster, en C
Mémoire	variables seulement	`malloc`, `free`, CHR RAM
Bibliothèque C	—	`string.h`, `stdlib.h`, `stdio.h`…

Choisir mc

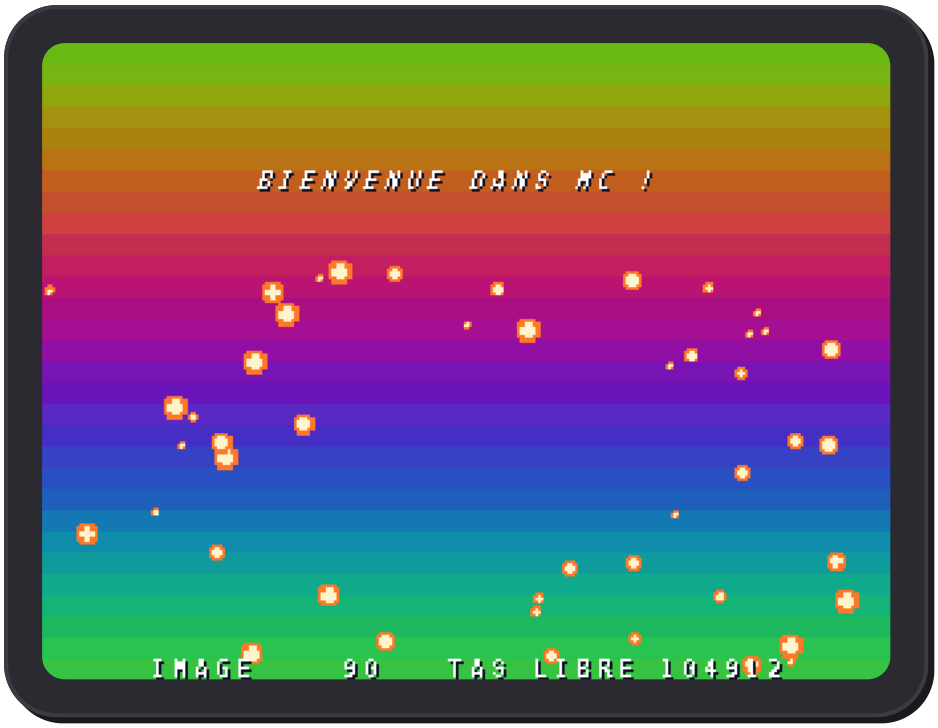
Une seule ligne dans `project.txt` suffit, et le programme inclut `mc.h` au lieu de `easy.h`. Toutes les fonctions commencent par `mc_`. On ne mélange pas les deux bibliothèques dans un même programme.

LISTING 11_mc/project.txt

```
# Chapter 11: first program with the low-level mc SDK
name    = mcdemo
title   = PREMIERS PAS AVEC MC
library = mc
sources = main.c
```

Le squelette ressemble à celui d'`easy` : `mc_init()` au début, puis une boucle qui commence par `mc_frame()`. Cette fonction attend l'image suivante, puis envoie les sprites à la console, lit les manettes et fait avancer les mélodies.

Un premier programme



Un arc-en-ciel fait de 30 effets raster, un titre qui ondule et 48 balles dessinées par le programme.

LISTING 11_mc/main.c

```c
// Chapter 11: a first program with the low-level "mc" SDK.
// Raster rainbow, a waving title (per-line scrolling), 48 bouncing balls drawn in CHR RAM,
// heap memory and snprintf.
#include "mc.h"
#include <stdio.h>
#include <stdlib.h>

typedef struct { int x, y, vx, vy; } ball_t;      // positions in 1/256 pixel
static ball_t *balls;
#define BALLS 48

static void rainbow(void) {                       // raster effect, called every 8 lines
  int a = mc_raster_line * 4;                     // angle: one turn (1024) over the screen height
  mc_color(4095, MC_RGB(110 + (mc_sin(a) * 90 >> 16), 110 + (mc_sin(a + 341) * 90 >> 16),
                        110 + (mc_sin(a + 683) * 90 >> 16)));
}

static void make_ball(void) {                     // an 8 x 8 disc, 4 bits per pixel, in CHR RAM
  uint8_t cell[32] = {0};
  for (int y = 0; y < 8; y++)
    for (int x = 0; x < 8; x++) {
      int dx = 2 * x - 7, dy = 2 * y - 7, d = dx * dx + dy * dy;
      int colour = d < 12 ? 2 : d < 50 ? 1 : 0;   // 2: highlight, 1: body, 0: transparent
      cell[y * 4 + x / 2] |= (uint8_t)(colour << (4 * (x & 1)));
    }
  mc_chr_upload(0, cell, sizeof cell);
  mc_sprite_sheet(MC_CHR(0));
  mc_color(16 * 16 + 1, MC_RGB(255, 120, 40));    // palette 16: the balls
  mc_color(16 * 16 + 2, MC_RGB(255, 240, 200));
}

int main(void) {
  mc_init();
  mc_text_layer(0, 3);
  mc_layer_linescroll(0, true);                   // each line of the text layer can move
  mc_background(4095);
```

```
  for (int y = 0; y < 240; y += 8) mc_raster_add(y, rainbow);
  make_ball();
  balls = calloc(BALLS, sizeof(ball_t));          // memory taken from the heap
  for (int i = 0; i < BALLS; i++)
    balls[i] = (ball_t){(int)(mc_rand() % 312) << 8, (80 + (int)(mc_rand() % 152)) << 8,
                        (int)(mc_rand() % 512) - 256, (int)(mc_rand() % 512) - 256};
  mc_sprites_used(BALLS);
  mc_display(DISP_ON | DISP_LAYER(0) | DISP_SPRITES);
  mc_text(0, 10, 6, "BIENVENUE DANS MC !", 0);
  char line[41];
  for (;;) {
    mc_frame();
    for (int y = 40; y < 64; y++)                 // the title waves: one offset per line
      mc_linescroll(0)[y].dx = (int16_t)(mc_sin((int)mc_frames * 8 + y * 6) * 10 >> 16);
    for (int i = 0; i < BALLS; i++) {
      ball_t *b = &balls[i];
      b->x += b->vx; b->y += b->vy;
      if (b->x < 0 || b->x > (312 << 8)) b->vx = -b->vx;
      if (b->y < (80 << 8) || b->y > (232 << 8)) b->vy = -b->vy;
      int zoom = 0x100 + (mc_sin((int)mc_frames * 6 + i * 21) >> 9);   // 0.5x to 1.5x
      mc_sprite(i, b->x >> 8, b->y >> 8, 0, 1, 1, 16, 4);
      mc_sprite_zoom(i, zoom, zoom);
    }
    snprintf(line, sizeof line, "IMAGE %5u   TAS LIBRE %6u", (unsigned)mc_frames,
(unsigned)mc_heap_free());
    mc_text(0, 5, 29, line, 0);
  }
}
```

Les effets raster

La console fabrique l'image ligne par ligne. `mc_raster_add(ligne, fonction)` demande d'appeler une fonction juste avant qu'une ligne soit dessinée : ce qu'elle change (une couleur, un défilement) s'applique à partir de cette ligne, au pixel près. Ici, 30 effets, un toutes les 8 lignes, changent la couleur de fond : c'est l'arc-en-ciel. Dans la fonction, `mc_raster_line` dit quelle ligne est en cours. On peut en poser 64 par image ; ils restent en place d'une image à l'autre.

> **Le saviez-vous ?**
>
> L'Amiga avait un coprocesseur, le *Copper*, qui ne faisait que cela : changer des registres à des lignes précises. Les fameuses « barres copper » des démos en viennent. Le premier écran d'accueil de la console (`examples/boot`) en est un hommage.

Le déflement ligne par ligne

`mc_layer_linescroll(plan, true)` autorise un décalage différent pour chaque ligne d'un plan ; `mc_linescroll(plan)[y].dx` est le décalage horizontal de la ligne `y`. En le faisant varier en sinus, le titre ondule comme un drapeau. Le même procédé donne les routes des jeux de course, les vagues ou les sols en perspective.

Dessiner dans la CHR RAM

Les dessins ne viennent pas forcément d'une image PNG. La **CHR RAM**, 128 Ko de dessins modifiables, peut recevoir ce que le programme calcule : `make_ball()` trace un disque pixel par pixel, l'envoie avec `mc_chr_upload`, et `MC_CHR(0)` donne l'adresse à utiliser comme planche de sprites. Un pixel occupe 4 bits : deux pixels par octet, celui de gauche dans les 4 bits de poids faible, 32 octets par case de 8 x 8.

Les sprites

`mc_sprite(n, x, y, case, largeur, hauteur, palette, priorité)` place le sprite `n` (0 à 511). La largeur et la hauteur se comptent en cases de 8 pixels ; `mc_sprite_zoom` l'agrandit ou le réduit (256 = taille normale). Les sprites sont d'abord rangés dans une copie en mémoire, que `mc_frame()` envoie d'un coup par DMA : on ne voit jamais un sprite à moitié déplacé. `mc_sprites_used(48)` dit à la console de ne parcourir que les 48 premiers.

La mémoire et la bibliothèque C

Avec `mc`, les fichiers d'en-tête habituels du C sont disponibles. `calloc` réserve ici la place des 48 balles dans le **tas**, la mémoire libre entre les variables et la pile (environ 100 Ko) ; `free` la rend. `snprintf` fabrique une ligne de texte avec des nombres, et `printf` écrit sur la console de debug du simulateur.

En-tête	Ce qu'il apporte
`<string.h>`	copier, comparer, chercher : `strcpy`, `strcmp`, `strlen`, `memcpy`…
`<stdlib.h>`	`malloc`, `calloc`, `realloc`, `free`, `abs`, `atoi`, `rand`, `qsort`
`<stdio.h>`	`snprintf`, `sprintf`, et `printf` vers la console de debug
`<ctype.h>`	`isdigit`, `toupper`…
`<assert.h>`	`assert` : arrête le programme avec un message si une condition est fausse

Pas de nombres à virgule

Le processeur de la console ne calcule qu'en nombres entiers. Le compilateur accepte `float`, mais l'imite très lentement. `mc` fournit la **virgule fixe** : un `fix16` est un entier qui compte en 65 536e, `FIX(1.5)` vaut 1,5, et `mc_sin`, `mc_cos`, `mc_atan2` travaillent avec des angles de 1024 pas par tour. Dans l'exemple, les positions des balles comptent en 256e de pixel, ce qui permet des vitesses fines.

Quatre manettes, l'analogique et le clavier

Avec `mc`, `mc_pad(j)` lit les quatre joueurs, comme `held` et `pressed` d'`easy`. La console lit aussi les sticks analogiques (`mc_stick_x`, `mc_stick_y` : stick 0 à gauche, 1 à droite, de −127 à +127, y positif vers le bas), les paddles (`mc_paddle`, de 0 à 255) et les spinners (`mc_spinner_delta` : le déplacement depuis l'image précédente), ainsi qu'un clavier USB. Ces entrées sont arrivées avec la **version 2** du matériel : un jeu qui en a besoin vérifie `mc_hw_version()`, qui vaut 1 sur un core plus ancien, où ces fonctions renvoient 0.

```
if (mc_hw_version() >= 2) {
  for (int j = 1; j <= 4; j++) {
    ship[j - 1].x += mc_stick_x(j, 0) / 32;        // left stick, -127 to +127
    if (mc_pad_pressed(j) & IN_B(1)) fire(j - 1);
  }
  if (mc_key_pressed(KEY_ESC)) paused = !paused;    // state of a key
  int e;
  while ((e = mc_key_event()) >= 0)                 // events, in order
    if (e & 0x200) type_key(e & 0x1FF);             // 0x200: key down
}
```

Une touche se désigne par son code PS/2, `KEY_xxx` dans `monocore.h`. Ce code repère une **position** sur le clavier et porte le nom de la touche d'un clavier QWERTY : sur un clavier AZERTY, la touche A est `KEY_Q`. `mc_key` et `mc_key_pressed` donnent l'état relevé par `mc_frame()`, pratique pour un jeu ; `mc_key_event()` rend les appuis et relâchements un par un, dans l'ordre, ce qui convient à la saisie de texte. L'exemple `examples/test_inputs` affiche tout ce que la console reçoit.

D'autres exemples

Le dossier `examples` du projet contient d'autres programmes écrits avec `mc` : `horizon`, une course en fausse 3D à la OutRun faite d'une seule texture de route décalée ligne par ligne ; `aube`, l'écran d'accueil, dont le lever de soleil n'est qu'une animation de palette, accompagné d'une musique orchestrale jouée par deux voix PCM ; `boot`, le premier écran d'accueil, avec ses barres copper.

Toutes les fonctions sont rassemblées dans l'annexe D ; le fichier `sdk/SDK_MC.md` donne en plus les règles du matériel à connaître pour adapter un jeu existant.

12

Projet : NEBULA

Un shoot'em up complet, dont tous les dessins sont calculés au démarrage.



L'écran titre : des lettres de la police agrandies quatre fois.



Une partie : soucoupes, tirs et traînée du réacteur.

Le vaisseau traverse un paysage de montagnes et de collines ; des vagues de soucoupes arrivent en ondulant, et des orbes tirent vers le joueur. Le bouton 1 tire, la croix pilote. Le programme (505 lignes, dans `examples/nebula`) n'utilise **aucun fichier image ni son** : les sprites sont dessinés en caractères dans le code, et les décors sont calculés.

Organisation du programme

Fonction	Rôle
`make_sprites`	convertit les dessins en caractères (`art_ship`...) en cases de sprites, dans la CHR RAM
`build_layer`	découpe une image calculée de 512 x 256 pixels en tuiles, en partageant les tuiles identiques
`make_backgrounds`	calcule les profils des montagnes et des collines, construit les trois plans de décor
`scroll_backgrounds`	fait défiler les trois plans à trois vitesses, et le sol en perspective
`sky`	effet raster : une couleur de ciel toutes les 4 lignes
`play_frame`	une image de jeu : manette, tirs, vagues d'ennemis, collisions
`fire_at_player`	tir d'un orbe, dirigé vers le joueur avec `mc_atan2`
`explosion`	lance des dizaines de particules dans toutes les directions

Fonction	Rôle
`update_fx`	fait vivre les particules : elles ralentissent, refroidissent et rétrécissent
`title_screen`	écran titre
`game_over`	fin de partie, avec le record gardé en mémoire sauvegardée (NVRAM)

Des dessins en caractères

Chaque sprite est dessiné dans le code, une chaîne de 16 caractères par ligne : un point est transparent, une lettre de `A` à `O` désigne une couleur de 1 à 15. `pack_art` range ces pixels en cases de 8 x 8, dans l'ordre que la console attend pour un sprite de 2 x 2 cases : en haut à gauche, en haut à droite, en bas à gauche, en bas à droite.

LISTING examples/nebula/main.c : le dessin du vaisseau

```
static const char *const art_ship[16] = {
  "................", "................", "..AA............", "..ABA...........",
  "...ABBA.........", "...ACCBBAA......", "DDACCCCCBBBAA...", "EEACCCCFFFCCBBA.",
  "EEACCCCFFFCCBBA.", "DDACCCCCBBBAA...", "...ACCBBAA......", "...ABBA.........",
  "..ABA...........", "..AA............", "................", "................"};
  ...
```

Des décors calculés

Les montagnes sont une somme de trois sinus : `far_top[x]` donne la hauteur du sommet pour chaque colonne. La fonction `far_pixel(x, y)` dit la couleur de chaque point d'une image de 512 x 256 pixels : rien au-dessus du sommet, une ligne claire sur la crête, puis des bandes de plus en plus sombres. `build_layer` découpe cette image en 2 048 tuiles de 8 x 8, mais n'en garde qu'une centaine : les tuiles identiques (le ciel vide, les bandes pleines) sont partagées. Pour les reconnaître vite, chaque tuile reçoit une empreinte (un *hachage*), et on ne compare en détail que les tuiles de même empreinte.

Pourquoi des bandes de 16 lignes ?

Les bandes de couleur commencent sur des multiples de 16 lignes, donc sur des bords de tuiles. Toutes les tuiles à l'intérieur d'une bande sont alors identiques : une seule suffit pour toute la bande.

Trois plans et un sol

Les étoiles (plan 2) défilent d'un pixel toutes les 8 images, les montagnes (plan 1) toutes les 4 images, les collines (plan 3) d'un pixel par image : c'est la **parallaxe**. Les 24 dernières lignes, le damier du sol, défilent de plus en plus vite vers le bas grâce au défilement ligne par ligne : on croit voir un sol qui fuit vers l'horizon. Les priorités rangent le tout : le texte (plan 0, priorité 3) devant les sprites (priorité 4), devant les collines (priorité 2), devant les montagnes (priorité 1), devant les étoiles (priorité 0).

La règle des priorités

Chaque pixel d'un plan a une priorité *effective* égale à deux fois celle du plan, plus 1 si la tuile porte la marque `T_PRIO`. Un sprite de priorité *p* passe devant les pixels de priorité effective inférieure ou égale à *p*. Le texte, de priorité effective 6, cache donc les sprites de priorité 4.

Viser le joueur

`mc_atan2(dy, dx)` donne la direction qui va d'un orbe au vaisseau, en 1024 pas par tour. `mc_cos` et `mc_sin` la transforment en une vitesse horizontale et verticale. Les positions comptent en 256e de pixel (`FP` vaut 8) : le tir avance d'environ 1,6 pixel par image, dans n'importe quelle direction.

Des particules

Une explosion lance jusqu'à 48 particules. Leur réserve de 160 places est prise dans le tas avec `calloc`. À chaque image, une particule ralentit (sa vitesse est multipliée par 15/16), change de palette en refroidissant (blanc, jaune, orange, rouge sombre) et rétrécit grâce au zoom. Avec les tirs et les ennemis, le jeu peut afficher 233 sprites à la fois.

Le listing complet

LISTING examples/nebula/project.txt

```
# NEBULA: shoot'em up example of the low-level "mc" SDK (all graphics generated at start-up)
name    = nebula
title   = NEBULA
library = mc
sources = main.c
```

LISTING examples/nebula/main.c

```
// NEBULA - a horizontal shoot'em up for MONOCORE-1, written with the low-level "mc" SDK.
//
// Shows what mc adds to the console's C programs:
//   - every graphic is generated at start-up in CHR RAM (no image file): sprites from ASCII art,
//     three background layers from procedural height profiles (mc_chr_upload);
//   - parallax on three layers, a perspective floor with per-line scrolling (mc_linescroll);
//   - a sky gradient made of 54 raster effects (mc_raster_add);
//   - up to 240 sprites a frame through the shadow sprite table (mc_sprite, mc_frame);
//   - zoomed sprites (title letters, explosion particles), aimed enemy shots (mc_atan2, mc_cos);
//   - heap memory (calloc), snprintf, PSG sound effects and music (mc_psg, mc_tune).
#include "mc.h"
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

extern const uint8_t font_chr[], font_pal[];   // SDK font, always added by build.py

// ---------------------------------------------------------------- memory map of CHR RAM
#define CHR_SPR    0x0000                  // sprite cells
#define CHR_STARS  0x1000                  // layer 2: stars
#define CHR_FAR    0x2000                  // layer 1: far mountains (384 tiles at most)
#define CHR_NEAR   0x5000                  // layer 3: near hills and floor (384 tiles at most)
#define CHR_FONT   0x8000                  // copy of the SDK font, for the title letters
#define CHR(off)   MC_CHR(off)
#define CELL_FONT  (CHR_FONT / 32)

// sprite cells (8 x 8 each); a 16 x 16 sprite uses 4 cells in the order tile + cy * 2 + cx
#define CELL_SHIP   0
#define CELL_SAUCER 4
#define CELL_ORB    8
#define CELL_LASER  12
#define CELL_SHOT   13
#define CELL_SPARK  14

// palettes
#define PAL_HUD    1                       // 1-3: recoloured SDK font
```

```
#define PAL_LETTER 4                    // 4-9: title letters
#define PAL_SPR    16
#define PAL_FX     17                   // 17-20: particles, from hot to cold
#define PAL_STARS  24
#define PAL_FAR    25
#define PAL_NEAR   26
#define SKY        4095                 // background colour entry, changed line by line

// sprite slots
#define S_PLAYER   0
#define S_LASER    1
#define N_LASER    16
#define S_ENEMY    17
#define N_ENEMY    24
#define S_SHOT     41
#define N_SHOT     32
#define S_FX       73
#define N_FX       160
#define S_USED     (S_FX + N_FX)

#define FP 8                            // positions in 8.8 fixed point
#define PX(v) ((v) >> FP)

// ---------------------------------------------------------------- sprite art
// '.' = transparent, 'A' to 'O' = colours 1 to 15 of PAL_SPR (or of PAL_FX for the spark)
static const char *const art_ship[16] = {
  "................", "................", "..AA............", "..ABA...........",
  "...ABBA.........", "...ACCBBAA......", "DDACCCCCBBBAA...", "EEACCCCFFFCCBBA.",
  "EEACCCCFFFCCBBA.", "DDACCCCCBBBAA...", "...ACCBBAA......", "...ABBA.........",
  "..ABA...........", "..AA............", "................", "................"};
static const char *const art_saucer[16] = {
  "................", "................", "................", "......GGGG......",
  "....GHHHHHHG....", "...GHHIHHIHHG...", "..GJJJJJJJJJJG..", ".GJKKJKKJKKJKJG.",
  "GJJJJJJJJJJJJJJG", ".GGJJJJJJJJJJGG.", "...GGGGGGGGGG...", "....I..I..I..I..",
  "................", "................", "................", "................"};
static const char *const art_orb[16] = {
  "................", ".....LLLLLL.....", "...LLMMMMMMLL...", "..LMMMMMMMMML..",
  ".LMMMNNNNNNMMML.", ".LMMNNOOOONNMML.", "LMMMNOOLLOONMMML", "LMMMNOLLLLONMMML",
  "LMMMNOLLLLONMMML", "LMMMNOOLLOONMMML", ".LMMNNOOOONNMML.", ".LMMMNNNNNNMMML.",
  "..LMMMMMMMMML..", "...LLMMMMMLL...", ".....LLLLLL.....", "................"};
static const char *const art_laser[8] = {
  "........", "........", "........", "EKKKKKKE", "EKKKKKKE", "........", "........", "........"};
static const char *const art_shot[8] = {
  "........", "..NNNN..", ".NNOONN.", ".NOOOON.", ".NOOOON.", ".NNOONN.", "..NNNN..", "........"};
static const char *const art_spark[8] = {
  "........", "........", "...BB...", "..BAAB..", "..BAAB..", "...BB...", "........", "........"};

static int art_colour(char c) { return (c >= 'A' && c <= 'O') ? c - 'A' + 1 : 0; }

// packs an ASCII picture (w x h pixels, multiples of 8) into 4 bpp cells, in sprite order
static void pack_art(uint8_t *dst, const char *const *rows, int w, int h) {
  int cw = w / 8, ch = h / 8;
  for (int cy = 0; cy < ch; cy++)
    for (int cx = 0; cx < cw; cx++) {
      uint8_t *cell = dst + (cy * cw + cx) * 32;
      for (int y = 0; y < 8; y++)
        for (int x = 0; x < 8; x += 2) {
          const char *r = rows[cy * 8 + y] + cx * 8 + x;
          cell[y * 4 + x / 2] = (uint8_t)(art_colour(r[0]) | (art_colour(r[1]) << 4));
        }
    }
}

static void make_sprites(void) {
  static uint8_t sheet[16 * 32];
  memset(sheet, 0, sizeof sheet);
  pack_art(sheet + CELL_SHIP * 32, art_ship, 16, 16);
  pack_art(sheet + CELL_SAUCER * 32, art_saucer, 16, 16);
  pack_art(sheet + CELL_ORB * 32, art_orb, 16, 16);
```

```
  pack_art(sheet + CELL_LASER * 32, art_laser, 8, 8);
  pack_art(sheet + CELL_SHOT * 32, art_shot, 8, 8);
  pack_art(sheet + CELL_SPARK * 32, art_spark, 8, 8);
  mc_chr_upload(CHR_SPR, sheet, sizeof sheet);
  mc_chr_upload(CHR_FONT, font_chr, 96 * 32);
  mc_sprite_sheet(CHR(CHR_SPR));
}

// ------------------------------------------------------------- procedural backgrounds
// A layer is a 512 x 256 picture given by a function colour(x, y); it is cut into 8 x 8 tiles,
// identical tiles are shared (hash then compare) and the tiles go to CHR RAM, the map to the layer.
typedef int (*pixel_fn)(int x, int y);
static uint8_t far_top[512], near_top[512];

static int far_pixel(int x, int y) {
  int top = far_top[x];
  if (y < top) return 0;
  if (y - top < 2) return 6;                        // lit ridge
  int band = (y - 128) / 16;                        // bands aligned on tiles: shared solid tiles
  return 1 + (band < 0 ? 0 : band > 4 ? 4 : band);
}

static int near_pixel(int x, int y) {
  if (y >= 216) return (((x >> 4) ^ (y >> 3)) & 1) ? 7 : 8;   // floor checkerboard
  int top = near_top[x];
  if (y < top) return 0;
  if (y - top < 2) return 6;
  int band = (y - 160) / 16;
  return 1 + (band < 0 ? 0 : band > 4 ? 4 : band);
}

static int build_layer(int layer, uint32_t chr_off, int max_tiles, pixel_fn colour, int pal, int prio) {
  uint8_t (*tiles)[32] = calloc((size_t)max_tiles, 32);
  uint32_t *hash = calloc((size_t)max_tiles, sizeof(uint32_t));
  uint32_t *map = malloc(64 * 32 * sizeof(uint32_t));
  if (!tiles || !hash || !map) { printf("build_layer: out of memory\n"); abort(); }
  int count = 1;                                    // tile 0 stays empty
  for (int ty = 0; ty < 32; ty++)
    for (int tx = 0; tx < 64; tx++) {
      uint8_t t[32];
      uint32_t h = 2166136261u, any = 0;
      for (int y = 0; y < 8; y++)
        for (int x = 0; x < 8; x += 2) {
          int a = colour(tx * 8 + x, ty * 8 + y), b = colour(tx * 8 + x + 1, ty * 8 + y);
          uint8_t v = (uint8_t)(a | (b << 4));
          t[y * 4 + x / 2] = v; any |= v; h = (h ^ v) * 16777619u;
        }
      int n = 0;
      if (any) {
        for (n = 1; n < count; n++) if (hash[n] == h && !memcmp(tiles[n], t, 32)) break;
        if (n == count) {
          if (count == max_tiles) { printf("build_layer: too many tiles\n"); abort(); }
          memcpy(tiles[count], t, 32); hash[count] = h; count++;
        }
      }
      map[ty * 64 + tx] = MC_TILE(n, pal, 0);
    }
  mc_chr_upload(chr_off, tiles, (uint32_t)count * 32);
  mc_layer(layer, CHR(chr_off), MC_TILE8 | MC_4BPP, 64, 32, prio);
  mc_tilemap_load(layer, map, 64 * 32);
  free(map); free(hash); free(tiles);
  return count;
}

static void make_stars(void) {
  static uint8_t t[4][32];
  memset(t, 0, sizeof t);
  t[1][3 * 4 + 1] = 0x01;                           // one dim pixel
  t[2][4 * 4 + 2] = 0x02;                           // one brighter pixel
  t[3][2 * 4 + 1] = 0x30; t[3][3 * 4 + 1] = 0x33; t[3][3 * 4 + 2] = 0x03; t[3][4 * 4 + 1]
```

```
= 0x30;   // small cross
176   mc_chr_upload(CHR_STARS, t, sizeof t);
177   mc_layer(2, CHR(CHR_STARS), MC_TILE8 | MC_4BPP, 64, 32, 0);
178   for (int y = 0; y < 32; y++)
179     for (int x = 0; x < 64; x++) {
180       uint32_t r = mc_rand() % 100;
181       int n = r < 6 ? 1 : r < 9 ? 2 : r < 10 ? 3 : 0;
182       mc_tile(2, x, y, MC_TILE(n, PAL_STARS, 0));
183     }
184 }
185
186 static void make_backgrounds(void) {
187   for (int x = 0; x < 512; x++) {                // periodic profiles: angle = 1024
steps per turn
188     int a = x * 2;
189     far_top[x]  = (uint8_t)(150 + ((mc_sin(a * 2) * 18 + mc_sin(a * 5) * 9 + mc_sin(a *
11) * 4) >> 16));
190     near_top[x] = (uint8_t)(186 + ((mc_sin(a * 3) * 13 + mc_sin(a * 7) * 7 + mc_sin(a *
17) * 3) >> 16));
191   }
192   int nf = build_layer(1, CHR_FAR, 384, far_pixel, PAL_FAR, 1);
193   int nn = build_layer(3, CHR_NEAR, 384, near_pixel, PAL_NEAR, 2);
194   make_stars();
195   mc_layer_linescroll(3, true);
196   printf("backgrounds: %d + %d tiles, heap free %u\n", nf, nn, (unsigned)mc_heap_free());
197 }
198
199 static void sky(void) {                          // raster effect: one sky colour every
4 lines
200   int k = mc_raster_line * 255 / 212;
201   mc_color(SKY, MC_RGB(8 + k * 112 / 255, 6 + k * 34 / 255, 30 + k * 80 / 255));
202 }
203
204 static void scroll_backgrounds(uint32_t t) {
205   int far = (int)(t >> 2), near = (int)t, stars = (int)(t >> 3);
206   mc_layer_scroll(2, stars, 0);
207   mc_layer_scroll(1, far, 0);
208   mc_layer_scroll(3, near, 0);
209   volatile linescroll_t *ls = mc_linescroll(3);
210   for (int y = 216; y < 240; y++) ls[y].dx = (int16_t)(near * (y - 214) / 6);   //
perspective floor
211   int tw = 140 + (mc_sin((int)t * 12) * 100 >> 16);                              //
twinkling stars
212   mc_color(PAL_STARS * 16 + 3, MC_RGB(tw, tw, 255));
213 }
214
215 // ------------------------------------------------------------------ palettes and text
216 static void set_rgb(int pal, int i, int r, int g, int b) { mc_color(pal * 16 + i,
MC_RGB(r, g, b)); }
217 static int font_white;
218
219 static void make_palettes(void) {
220   static const uint8_t spr[16][3] = {
221     {0, 0, 0}, {20, 30, 90}, {60, 120, 230}, {200, 210, 225}, {255, 140, 30}, {255, 230,
80},
222     {120, 240, 255}, {120, 20, 30}, {230, 50, 60}, {255, 220, 60}, {110, 110, 130}, {200,
200, 215},
223     {60, 20, 90}, {150, 60, 200}, {255, 90, 180}, {255, 255, 255}};
224   for (int i = 1; i < 16; i++) set_rgb(PAL_SPR, i, spr[i][0], spr[i][1], spr[i][2]);
225   static const uint8_t fx[4][2][3] = {
226     {{255, 255, 230}, {255, 220, 90}}, {{255, 220, 90}, {255, 130, 30}},
227     {{255, 130, 30}, {200, 40, 20}}, {{170, 40, 20}, {80, 20, 30}}};
228   for (int k = 0; k < 4; k++) {
229     set_rgb(PAL_FX + k, 1, fx[k][0][0], fx[k][0][1], fx[k][0][2]);
230     set_rgb(PAL_FX + k, 2, fx[k][1][0], fx[k][1][1], fx[k][1][2]);
231   }
232   set_rgb(PAL_STARS, 1, 90, 90, 140); set_rgb(PAL_STARS, 2, 160, 160, 210);
set_rgb(PAL_STARS, 3, 230, 230, 255);
233   static const uint8_t far[7][3] = {{0, 0, 0}, {90, 60, 140}, {75, 50, 120}, {60, 40,
100}, {48, 32, 84}, {36, 24, 66}, {160, 120, 210}};
234   for (int i = 1; i < 7; i++) set_rgb(PAL_FAR, i, far[i][0], far[i][1], far[i][2]);
235   static const uint8_t near[9][3] = {{0, 0, 0}, {20, 70, 80}, {16, 58, 68}, {12, 46, 56},
{9, 36, 46}, {6, 26, 36},
236                                      {70, 170, 150}, {28, 36, 66}, {48, 60, 100}};
237   for (int i = 1; i < 9; i++) set_rgb(PAL_NEAR, i, near[i][0], near[i][1], near[i][2]);
```

```
  // recoloured copies of the SDK font palette (palette 0, set by mc_text_layer)
  font_white = 1;
  for (int i = 1; i < 16; i++) if (PALETTE[i] == MC_RGB(255, 255, 255)) font_white = i;
  static const uint8_t ink[9][3] = {{255, 220, 60}, {90, 220, 255}, {255, 110, 200},
                                    {255, 80, 80}, {255, 170, 40}, {255, 240, 80}, {90,
240, 110}, {80, 200, 255}, {200, 110, 255}};
  for (int p = 0; p < 9; p++) {
    for (int i = 0; i < 16; i++) PALETTE[(PAL_HUD + p) * 16 + i] = PALETTE[i];
    set_rgb(PAL_HUD + p, font_white, ink[p][0], ink[p][1], ink[p][2]);
  }
}

static void text(int col, int row, const char *s, int pal) { mc_text(0, col, row, s,
pal); }
static void text_center(int row, const char *s, int pal) { text((40 - (int)strlen(s)) /
2, row, s, pal); }
static void text_clear(void) { mc_text_clear(0); }

// ------------------------------------------------------------------ sound
static int sfx_shot, sfx_boom, sfx_boom_freq;

static void sfx_update(void) {
  if (sfx_shot > 0) { sfx_shot--; mc_psg(2, PSG_SQUARE, 700 + sfx_shot * 220, 96,
sfx_shot * 9, sfx_shot * 9); }
  if (sfx_boom > 0) { sfx_boom -= 6; if (sfx_boom < 0) sfx_boom = 0; mc_psg(3, PSG_NOISE,
(uint32_t)sfx_boom_freq, 0, sfx_boom, sfx_boom); }
}
static void sfx_explode(int big) { sfx_boom = big ? 230 : 150; sfx_boom_freq = big ? 2500
: 6000; }

static const char *tune_lead =
  "A4 - C5 E5 - D5 C5 - B4 - G4 - A4 - - . E5 - G5 A5 - G5 E5 - D5 - C5 - B4 - - . ";
static const char *tune_bass =
  "A2 A3 A2 A3 A2 A3 A2 A3 F2 F3 F2 F3 G2 G3 G2 G3 C3 C4 C3 C4 C3 C4 C3 C4 G2 G3 G2 G3 E2
E3 E2 E3 ";

static void music(bool on) {
  if (on) { mc_tune(0, tune_lead, 8, 40); mc_tune(1, tune_bass, 8, 55); }
  else { mc_tune_stop(0); mc_tune_stop(1); }
}

// ------------------------------------------------------------------ game objects
typedef struct { bool on; int x, y, vx, vy; } shot_t;
typedef struct { bool on; int type, x, y, base_y, phase, hp, t, flash; } enemy_t;
typedef struct { int x, y, vx, vy, life; } particle_t;

static shot_t lasers[N_LASER], shots[N_SHOT];
static enemy_t enemies[N_ENEMY];
static particle_t *fx;                              // N_FX particles, from the heap
static int fx_next;
static int px, py, lives, invuln, cooldown;
static uint32_t score, best;

static void spr16(int n, int x, int y, int cell, int pal) {
  mc_sprite(n, x, y, cell, 2, 2, pal, 4); mc_sprite_zoom(n, 0x100, 0x100);
}
static void spr8(int n, int x, int y, int cell, int pal) {
  mc_sprite(n, x, y, cell, 1, 1, pal, 4); mc_sprite_zoom(n, 0x100, 0x100);
}

static void spark(int x, int y, int speed, int life) {     // x, y in pixels; speed in
1/256 px
  particle_t *p = &fx[fx_next]; fx_next = (fx_next + 1) % N_FX;
  int a = (int)(mc_rand() & 1023), s = speed / 2 + (int)(mc_rand() % (uint32_t)(speed +
1));
  p->x = x << FP; p->y = y << FP;
  p->vx = (mc_cos(a) * s) >> 16; p->vy = (mc_sin(a) * s) >> 16;
  p->life = life / 2 + (int)(mc_rand() % (uint32_t)(life / 2 + 1));
}
static void explosion(int x, int y, int n, int speed) { for (int i = 0; i < n; i++)
spark(x, y, speed, 40); }

static void update_fx(void) {
  for (int i = 0; i < N_FX; i++) {
    particle_t *p = &fx[i];
```

```
    if (p->life <= 0) { mc_sprite_hide(S_FX + i); continue; }
    p->life--; p->x += p->vx; p->y += p->vy;
    p->vx = p->vx * 15 / 16; p->vy = p->vy * 15 / 16;   // drag
    int stage = p->life > 30 ? 0 : p->life > 20 ? 1 : p->life > 10 ? 2 : 3;
    int z = 0x80 + p->life * 8;                          // particles shrink as they cool
    mc_sprite(S_FX + i, PX(p->x) - 4, PX(p->y) - 4, CELL_SPARK, 1, 1, PAL_FX + stage, 4);
    mc_sprite_zoom(S_FX + i, z, z);
  }
}

static void reset_objects(void) {
  memset(lasers, 0, sizeof lasers); memset(shots, 0, sizeof shots); memset(enemies, 0,
sizeof enemies);
  memset(fx, 0, N_FX * sizeof(particle_t));
  mc_sprites_hide_all();
}

static void spawn_enemy(int type, int x, int y, int phase) {
  for (int i = 0; i < N_ENEMY; i++)
    if (!enemies[i].on) {
      enemy_t *e = &enemies[i];
      e->on = true; e->type = type; e->x = x << FP; e->y = e->base_y = y << FP;
      e->phase = phase; e->hp = type ? 4 : 1; e->t = (int)(mc_rand() % 60); e->flash = 0;
      return;
    }
}

static void fire_at_player(int ex, int ey) {      // aimed shot: angle from mc_atan2
  for (int i = 0; i < N_SHOT; i++)
    if (!shots[i].on) {
      int a = mc_atan2((py + 8) - ey, (px + 8) - ex);
      shots[i] = (shot_t){true, ex << FP, ey << FP, (mc_cos(a) * 420) >> 16, (mc_sin(a) *
420) >> 16};
      return;
    }
}

static bool hit(int ax, int ay, int aw, int ah, int bx, int by, int bw, int bh) {
  return ax < bx + bw && bx < ax + aw && ay < by + bh && by < ay + ah;
}

static void hurt_player(void) {
  if (invuln) return;
  explosion(px + 8, py + 8, 40, 700);
  sfx_explode(1);
  lives--; invuln = 120;
}

// ------------------------------------------------------------------ one frame of play
static void hud(void) {
  char line[48];
  snprintf(line, sizeof line, "SCORE %06u", (unsigned)score);   text(1, 0, line,
PAL_HUD);
  snprintf(line, sizeof line, "VIES %d", lives < 0 ? 0 : lives); text(16, 0, line,
PAL_HUD + 1);
  snprintf(line, sizeof line, "RECORD %06u", (unsigned)best);   text(26, 0, line, PAL_HUD
+ 2);
}

static void play_frame(uint32_t t) {
  uint32_t pad = mc_pad(1);
  if (pad & IN_LEFT)  px -= 2;
  if (pad & IN_RIGHT) px += 2;
  if (pad & IN_UP)    py -= 2;
  if (pad & IN_DOWN)  py += 2;
  px = px < 0 ? 0 : px > 300 ? 300 : px;
  py = py < 12 ? 12 : py > 196 ? 196 : py;

  // player lasers
  if (cooldown > 0) cooldown--;
  if ((pad & IN_B(1)) && !cooldown && lives > 0) {
    for (int i = 0; i < N_LASER; i++)
      if (!lasers[i].on) { lasers[i] = (shot_t){true, (px + 12) << FP, (py + 4) << FP, 7
<< FP, 0}; break; }
    cooldown = 7; sfx_shot = 6;
```

```
  }
  for (int i = 0; i < N_LASER; i++) {
    shot_t *s = &lasers[i];
    if (s->on) { s->x += s->vx; if (PX(s->x) > 320) s->on = false; }
    if (s->on) spr8(S_LASER + i, PX(s->x), PX(s->y), CELL_LASER, PAL_SPR); else
mc_sprite_hide(S_LASER + i);
  }

  // waves: five saucers every 100 frames, an orb every 260
  if (t % 100 == 0) { int y = 30 + (int)(mc_rand() % 130); for (int k = 0; k < 5; k++)
spawn_enemy(0, 330 + k * 22, y, k * 60); }
  if (t % 260 == 130) spawn_enemy(1, 330, 40 + (int)(mc_rand() % 130), 0);

  for (int i = 0; i < N_ENEMY; i++) {
    enemy_t *e = &enemies[i];
    if (!e->on) { mc_sprite_hide(S_ENEMY + i); continue; }
    e->t++;
    if (e->type == 0) { e->x -= 320; e->y = e->base_y + ((mc_sin(e->t * 8 + e->phase) *
26) >> 8); }
    else {
      e->x -= 128; e->y = e->base_y + ((mc_sin(e->t * 3) * 14) >> 8);
      if (e->t % 70 == 0 && PX(e->x) < 310) fire_at_player(PX(e->x) + 8, PX(e->y) + 8);
    }
    int ex = PX(e->x), ey = PX(e->y);
    if (ex < -20) { e->on = false; mc_sprite_hide(S_ENEMY + i); continue; }
    for (int j = 0; j < N_LASER; j++) {
      shot_t *s = &lasers[j];
      if (s->on && hit(PX(s->x), PX(s->y) + 3, 8, 2, ex + 1, ey + 3, 14, 10)) {
        s->on = false; e->hp--; e->flash = 4;
        if (e->hp <= 0) {
          explosion(ex + 8, ey + 8, e->type ? 48 : 18, e->type ? 900 : 600);
          sfx_explode(e->type); score += e->type ? 500 : 100;
          e->on = false; break;
        }
        spark(ex + 2, ey + 8, 300, 12);
      }
    }
    if (!e->on) { mc_sprite_hide(S_ENEMY + i); continue; }
    if (lives > 0 && hit(ex + 2, ey + 4, 12, 8, px + 3, py + 6, 10, 4)) hurt_player();
    if (e->flash) e->flash--;
    spr16(S_ENEMY + i, ex, ey, e->type ? CELL_ORB : CELL_SAUCER, e->flash ? PAL_FX :
PAL_SPR);
  }

  for (int i = 0; i < N_SHOT; i++) {
    shot_t *s = &shots[i];
    if (s->on) {
      s->x += s->vx; s->y += s->vy;
      int sx = PX(s->x), sy = PX(s->y);
      if (sx < -8 || sx > 328 || sy < -8 || sy > 248) s->on = false;
      else if (lives > 0 && hit(sx - 2, sy - 2, 4, 4, px + 4, py + 6, 8, 4)) { s->on =
false; hurt_player(); }
    }
    if (s->on) spr8(S_SHOT + i, PX(s->x) - 4, PX(s->y) - 4, CELL_SHOT, PAL_SPR); else
mc_sprite_hide(S_SHOT + i);
  }

  // player, blinking while invulnerable, with an engine trail
  if (invuln) invuln--;
  if (lives > 0 && !(invuln & 4)) spr16(S_PLAYER, px, py, CELL_SHIP, PAL_SPR); else
mc_sprite_hide(S_PLAYER);
  if (lives > 0 && (t & 1)) {
    particle_t *p = &fx[fx_next]; fx_next = (fx_next + 1) % N_FX;
    *p = (particle_t){(px + 1) << FP, (py + 8 + (int)(mc_rand() % 3) - 1) << FP, -(300 +
(int)(mc_rand() % 200)), 0, 14};
  }
  update_fx();
  if (score > best) best = score;
}

// ------------------------------------------------------------------ screens
static void save_best(void) {                    // NVRAM: 32 KB, not yet saved to the
SD card by the core
  memcpy((void *)MC_NVRAM, "NBLA", 4);
  for (int i = 0; i < 4; i++) MC_NVRAM[4 + i] = (uint8_t)(best >> (8 * i));
```

```
}
static void load_best(void) {
  best = 5000;
  if (!memcmp((const void *)MC_NVRAM, "NBLA", 4))
    best = MC_NVRAM[4] | (MC_NVRAM[5] << 8) | ((uint32_t)MC_NVRAM[6] << 16) |
((uint32_t)MC_NVRAM[7] << 24);
}

static void title_screen(void) {
  static const char name[] = "NEBULA";
  reset_objects(); text_clear();
  char line[48];
  text_center(15, "UN EXEMPLE DU SDK MC", PAL_HUD + 1);
  text_center(18, "CROIX : PILOTER   B1 : TIRER", 0);
  snprintf(line, sizeof line, "RECORD %06u", (unsigned)best); text_center(24, line,
PAL_HUD + 2);
  for (uint32_t t = 0;; t++) {
    mc_frame();
    scroll_backgrounds(mc_frames);
    for (int i = 0; i < 6; i++) {                // big letters: font cells zoomed x4,
on a wave
      int n = S_ENEMY + i;
      mc_sprite(n, 64 + i * 32, 40 + (mc_sin((int)t * 10 + i * 90) * 10 >> 16), CELL_FONT
+ name[i] - 32, 1, 1, PAL_LETTER + i, 4);
      mc_sprite_zoom(n, 0x400, 0x400);
    }
    text_center(21, (t / 30) % 2 ? "                  " : "APPUYEZ SUR START", PAL_HUD);
    if (mc_pad_pressed(1) & IN_START) break;
  }
}

static void game(void) {
  reset_objects(); text_clear();
  px = 40; py = 100; lives = 3; invuln = 90; cooldown = 0; score = 0;
  for (uint32_t t = 1; lives > 0 || invuln > 0; t++) {
    mc_frame();
    scroll_backgrounds(mc_frames);
    play_frame(t);
    sfx_update();
    hud();
  }
}

static void game_over(void) {
  text_center(11, "PARTIE TERMINEE", PAL_HUD + 3);
  if (score >= best) { save_best(); text_center(13, "NOUVEAU RECORD !", PAL_HUD); }
  for (int t = 0; t < 240; t++) {
    mc_frame();
    scroll_backgrounds(mc_frames);
    update_fx(); sfx_update();
    if (t > 60 && (mc_pad_pressed(1) & IN_START)) break;
  }
}

int main(void) {
  mc_init();
  mc_text_layer(0, 3);                           // layer 0: text, in front of
everything
  make_palettes();
  mc_color(SKY, 0);
  mc_background(SKY);
  text_center(14, "PREPARATION DES DECORS...", 0);
  mc_display(DISP_ON | DISP_LAYER(0));
  make_sprites();
  make_backgrounds();
  fx = calloc(N_FX, sizeof(particle_t));
  mc_sprites_used(S_USED);
  for (int y = 0; y <= 212; y += 4) mc_raster_add(y, sky);
  mc_display(DISP_ON | DISP_LAYER(0) | DISP_LAYER(1) | DISP_LAYER(2) | DISP_LAYER(3) |
DISP_SPRITES);
  load_best();
  music(true);
  for (;;) { title_screen(); game(); game_over(); }
}
```

Pour aller plus loin

1. Changez les dessins du vaisseau et des ennemis, ou les couleurs du ciel.
2. Ajoutez un bonus qui donne un tir double pendant quelques secondes.
3. Faites apparaître un « boss » : un grand sprite de 64 x 64 (8 x 8 cases) qui encaisse 30 tirs.
4. Remplacez les décors calculés par un paysage dessiné, converti avec `maps = …` dans `project.txt`.
5. Passez l'écran en 384 x 224 avec `mc_video_mode`, comme une borne d'arcade.

13

La haute résolution

640 pixels de large, et même 480 lignes : la place qu'il faut aux éditeurs et aux outils.

Depuis la version 5 de son matériel, la MONOCORE-1 sait afficher bien plus de détails que ses 320 x 240 habituels. Ce n'est pas fait pour tous les programmes : un jeu garde en général la **basse résolution**, ses gros pixels nets et son allure de borne d'arcade. La haute résolution sert surtout aux **éditeurs et aux outils** : beaucoup de texte, des grilles, des réglages, comme dans MONOTRACKER ou SOLFEGE.

Mode	Taille	Pour quoi faire
basse résolution	224 à 384 x 192 à 288 (320 x 240 au départ)	les jeux : gros pixels, tous les effets de la console
haute résolution	jusqu'à 640 de large, avec 270 lignes au plus (640 x 240, 640 x 270)	80 colonnes de texte en cases de 8 x 8 ; les pixels sont deux fois plus hauts que larges
entrelacé	640 x 480 (de 384 à 512 lignes)	une image fine dans les deux sens : des caractères de 8 x 16, des dessins précis

Il suffit de demander la taille voulue à `mc_video_mode` : une hauteur de 384 à 512 lignes choisit le mode entrelacé. Avant, le programme vérifie que la console sait le faire.

```
if (mc_hw_version() >= 5)
  mc_video_mode(640, 480);          // interlaced: two fields of 240 lines
else
  mc_video_mode(320, 240);          // an older core: stay in low resolution
```

L'entrelacé, comment ça marche

Un téléviseur dessine 60 images par seconde de 240 lignes. Pour en montrer 480, la console alterne deux **trames** : l'une affiche les lignes paires de l'image, l'autre les lignes impaires. Sur la MiSTer, le framework reconstitue l'image entière en HDMI. Pour le programme, presque rien ne change : les plans, les sprites et les tables de défilement se placent toujours en lignes de l'image complète (0 à 479). Seuls le compteur de lignes `VCOUNT`, les interruptions de ligne et les effets raster comptent en lignes de trame (0 à 239). `mc_interlaced()` dit si le mode est actif, `mc_field()` donne la parité de la trame en cours.

Pas de trait d'un seul pixel de haut

Une ligne horizontale d'un pixel n'apparaît que dans une trame sur deux : elle scintille. Donnez aux traits horizontaux au moins deux lignes d'épaisseur : bords de cadre, lignes de portée, et les barres des lettres. Doubler chaque ligne de la police 8 x 8 du SDK suffit à le garantir, c'est ce que fait l'exemple de ce chapitre. Les traits verticaux, eux, peuvent rester fins.

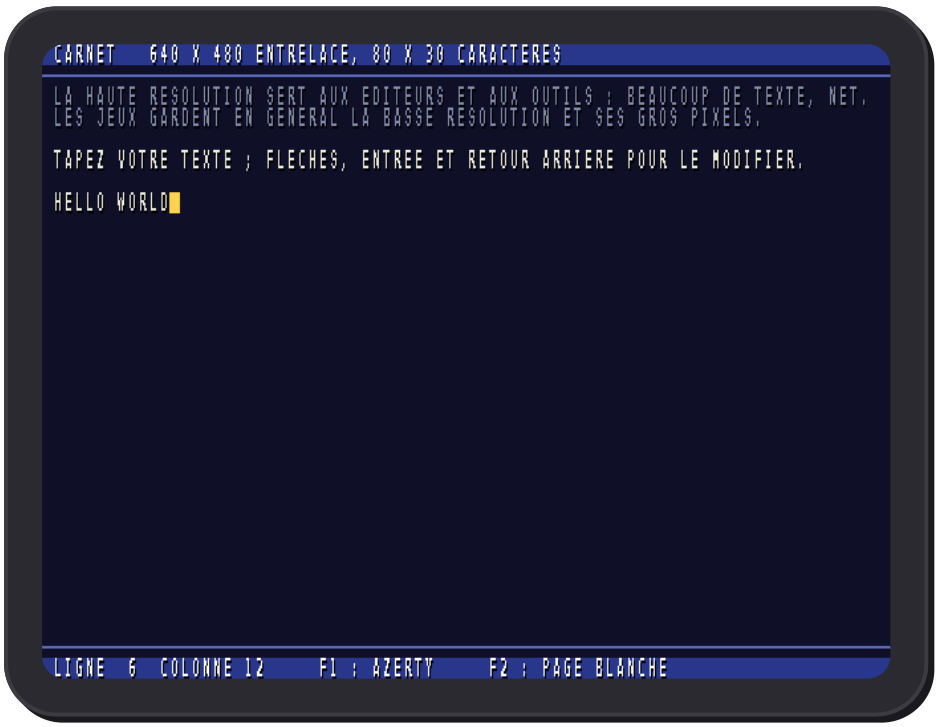
Couvrir 640 x 480 avec des tuiles

Une carte de tuiles compte au plus 4096 cases. En tuiles de 8 x 8, il en faudrait 80 x 60 = 4800 pour couvrir l'image : c'est trop. Deux solutions, que les programmes du projet emploient.

Solution	Comment	Exemples
tuiles de 16 x 16	une carte de 64 x 32 couvre 1024 x 512 pixels : tout l'écran d'un seul plan. Le texte tient alors en 40 x 30 caractères de 16 x 16	AUBE (une image en 8 bits par pixel), SOLFEGE
deux plans par couche	en tuiles de 8 x 8, une carte de 128 x 32 couvre 1024 x 256 pixels. Un plan prend les lignes du haut, un autre celles du bas ; le défilement par ligne cache, dans chaque plan, la moitié qui n'est pas la sienne	MONOTRACKER, l'exemple de ce chapitre : 80 x 30 caractères de 8 x 16

Un exemple : CARNET

CARNET est un petit bloc-notes : on tape du texte au clavier, sur 80 colonnes et 30 lignes. Son cadre utilise des tuiles de 16 x 16, son texte deux plans de tuiles de 8 x 8, et son curseur est un sprite.



CARNET : 80 x 30 caractères de 8 x 16, un cadre en tuiles de 16 x 16 et un curseur en sprite.

LISTING 13_hires/project.txt

```
# Chapter 13: high resolution, 640 x 480 interlaced (hardware version 5)
name    = carnet
title   = CARNET
library = mc
sources = main.c
```

LISTING 13_hires/main.c

```
  1 // Chapter 13: CARNET, a notepad in 640 x 480 interlaced (hardware version 5).
  2 // 80 x 30 characters of 8 x 16 pixels, typed on the keyboard. It shows the two ways to
cover 640 x 480:
  3 //  - layer 2: 16 x 16 tiles, a map of 64 x 32 (1024 x 512 pixels) covers the whole
```

```
picture (the frame);
//  - layers 0 and 1: 8 x 8 tiles for the text; a map of 128 x 32 covers only 1024 x 256 pixels, so
//    layer 0 holds the text rows 0-14 and layer 1 the rows 15-29, and the line scroll table of each
//    one shows two empty map rows in the half of the screen that is not its own.
#include "mc.h"
#include <stdio.h>
#include <string.h>

#define COLS 80
#define ROWS 30
#define TOP 2                           // the text area: rows 2 to 27, columns 1 to 78
#define BOTTOM 27
enum { P_TEXT, P_DIM, P_TITLE, P_FRAME = 8, P_CURSOR = 9 };
#define KEY_COMMA 0x41                  // keys without a name in monocore.h (PS/2 codes)
#define KEY_PERIOD 0x49
#define KEY_SEMICOLON 0x4C
#define KEY_MINUS 0x4E

static char page[ROWS][COLS + 1];       // the text typed
static int cx = 1, cy = TOP, azerty = 1;

// ------------------------------------------------------------------ the 8 x 16 font
// The 8 x 8 font of the SDK, each row drawn twice: character c uses the tiles 2c (top) and 2c + 1
// (bottom). Doubling the rows also keeps every horizontal stroke two lines thick: a line of a single
// pixel would show in one field out of two, and flicker.
static void make_font(void) {
  static uint8_t top[32], bot[32];
  for (int c = 0; c < 96; c++) {
    const uint8_t *g = &font_chr[c * 32];        // 8 rows of 4 bytes (4 bits a pixel)
    for (int y = 0; y < 8; y++) {
      memcpy(&top[y * 4], &g[(y / 2) * 4], 4);
      memcpy(&bot[y * 4], &g[(4 + y / 2) * 4], 4);
    }
    mc_chr_upload((2 * c) * 32, top, 32);
    mc_chr_upload((2 * c + 1) * 32, bot, 32);
  }
}
// a character at column x, row y: two tiles, in layer 0 (rows 0-14) or layer 1 (rows 15-29)
static void put(int x, int y, char ch, int pal) {
  int g = (ch < 32 || ch > 127) ? 0 : ch - 32;
  volatile uint32_t *m = mc_tilemap(y < 15 ? 0 : 1) + (y % 15) * 2 * 128 + x;
  m[0] = TILE(2 * g, pal);
  m[128] = TILE(2 * g + 1, pal);
}
static void print(int x, int y, const char *s, int pal) { for (; *s && x < COLS; s++) put(x++, y, *s, pal); }

// each text layer shows its own half; elsewhere its line scroll points at map rows 30-31, left empty
static void text_layers(void) {
  for (int l = 0; l < 2; l++) {
    mc_layer(l, MC_CHR(0), MC_TILE8 | MC_4BPP, 128, 32, 3);
    mc_layer_scroll(l, 0, l ? 16 : 0);           // layer 1: picture line 240 shows map row 0
    mc_layer_linescroll(l, true);
    volatile linescroll_t *ls = mc_linescroll(l);
    for (int y = 0; y < 480; y++) {              // y: line of the whole picture, also when interlaced
      bool mine = l ? y >= 240 : y < 240;
      ls[y].dx = 0;
      ls[y].dy = (int16_t)(mine ? 0 : (l ? 224 : 240) + (y & 15) - y);
    }
  }
}

// ------------------------------------------------------------------ the frame, in 16 x 16 tiles
// tile 0 of this layer: empty; 1: solid; 2 and 3: the top and bottom edges, two pixels thick
#define FRAME_OFF 0x4000
```

```
 67 static void make_frame(void) {
 68   static uint8_t t[128];
 69   memset(t, 0, 128); mc_chr_upload(FRAME_OFF, t, 128);
 70   memset(t, 0x11, 128); mc_chr_upload(FRAME_OFF + 128, t, 128);
 71   memset(t, 0, 128); memset(t + 7 * 8, 0x22, 16); mc_chr_upload(FRAME_OFF + 2 * 128, t,
128);  // rows 7-8
 72   mc_layer(2, MC_CHR(FRAME_OFF), MC_TILE16 | MC_4BPP, 64, 32, 1);
 73   volatile uint32_t *m = mc_tilemap(2);
 74   for (int x = 0; x < 40; x++) {
 75     m[0 * 64 + x] = TILE(1, P_FRAME);            // title bar: the first 16 lines
 76     m[1 * 64 + x] = TILE(2, P_FRAME);            // a line under it
 77     m[28 * 64 + x] = TILE(2, P_FRAME);           // a line above the status bar
 78     m[29 * 64 + x] = TILE(1, P_FRAME);           // status bar: the last 16 lines
 79   }
 80 }
 81
 82 // ------------------------------------------------------------- keyboard
 83 static int key_char(int k) {
 84   static const int16_t letters[26] = { KEY_A, KEY_B, KEY_C, KEY_D, KEY_E, KEY_F, KEY_G,
KEY_H, KEY_I, KEY_J, KEY_K, KEY_L, KEY_M,
 85                                        KEY_N, KEY_O, KEY_P, KEY_Q, KEY_R, KEY_S, KEY_T,
KEY_U, KEY_V, KEY_W, KEY_X, KEY_Y, KEY_Z };
 86   static const int16_t digits[10] = { KEY_0, KEY_1, KEY_2, KEY_3, KEY_4, KEY_5, KEY_6,
KEY_7, KEY_8, KEY_9 };
 87   if (azerty) {                                  // the letters that move on a French
keyboard
 88     if (k == KEY_Q) return 'A'; if (k == KEY_A) return 'Q';
 89     if (k == KEY_W) return 'Z'; if (k == KEY_Z) return 'W';
 90     if (k == KEY_SEMICOLON) return 'M'; if (k == KEY_M) return ',';
 91   }
 92   for (int i = 0; i < 26; i++) if (letters[i] == k) return 'A' + i;
 93   for (int i = 0; i < 10; i++) if (digits[i] == k) return '0' + i;
 94   if (k == KEY_SPACE) return ' ';
 95   if (k == KEY_PERIOD) return azerty ? ';' : '.';
 96   if (k == KEY_COMMA) return azerty ? ';' : ',';
 97   if (k == KEY_MINUS) return azerty ? ')' : '-';
 98   return 0;
 99 }
100 static void status(void) {
101   char s[COLS + 1];
102   snprintf(s, sizeof s, " LIGNE %2d  COLONNE %2d     F1 : %s     F2 : PAGE BLANCHE", cy -
TOP + 1, cx, azerty ? "AZERTY" : "QWERTY");
103   for (int x = 0; x < COLS; x++) put(x, ROWS - 1, ' ', P_TITLE);
104   print(0, ROWS - 1, s, P_TITLE);
105 }
106 static void type(int k) {
107   int c = key_char(k);
108   if (c && cx < COLS - 1) { page[cy][cx] = (char)c; put(cx, cy, (char)c, P_TEXT); cx++; }
109   else if (k == KEY_ENTER && cy < BOTTOM) { cy++; cx = 1; }
110   else if (k == KEY_BACKSPACE && cx > 1) { cx--; page[cy][cx] = ' '; put(cx, cy, ' ',
P_TEXT); }
111   else if (k == KEY_LEFT && cx > 1) cx--;
112   else if (k == KEY_RIGHT && cx < COLS - 2) cx++;
113   else if (k == KEY_UP && cy > TOP) cy--;
114   else if (k == KEY_DOWN && cy < BOTTOM) cy++;
115   else if (k == KEY_F1) azerty = !azerty;
116   else if (k == KEY_F2) {
117     for (int y = TOP; y <= BOTTOM; y++) for (int x = 1; x < COLS - 1; x++) { page[y][x] =
' '; put(x, y, ' ', P_TEXT); }
118     cx = 1; cy = TOP;
119   }
120   status();
121 }
122
123 int main(void) {
124   mc_init();
125   if (mc_hw_version() < 5) {                     // no 640 x 480 before version 5: say so
in 320 x 240
126     mc_color(P_TEXT * 16 + 1, MC_RGB(250, 210, 80));
127     make_font(); text_layers(); mc_tilemap_fill(0, 0);
128     mc_display(DISP_ON | DISP_LAYER(0));
129     print(2, 6, "CARNET DEMANDE LE CORE VERSION 5", P_TEXT);
130     for (;;) mc_frame();
131   }
132   mc_video_mode(640, 480);                       // a height of 384 to 512 selects the
```

```
interlaced mode
133   mc_color(P_TEXT * 16 + 1, MC_RGB(235, 235, 220));
134   mc_color(P_DIM * 16 + 1, MC_RGB(130, 140, 170));
135   mc_color(P_TITLE * 16 + 1, MC_RGB(255, 255, 255));
136   mc_color(P_FRAME * 16 + 1, MC_RGB(40, 60, 140));
137   mc_color(P_FRAME * 16 + 2, MC_RGB(90, 110, 190));
138   mc_color(P_CURSOR * 16 + 1, MC_RGB(250, 210, 80));
139   mc_color(4095, MC_RGB(16, 20, 40)); mc_background(4095);   // the colour behind
everything: an entry of the palette
140   make_font();
141   text_layers();
142   make_frame();
143   mc_tilemap_fill(0, 0); mc_tilemap_fill(1, 0);
144   // the cursor: a sprite, an 8 x 8 cell zoomed twice in height; sprites use picture
coordinates
145   static uint8_t block[32];
146   memset(block, 0x11, sizeof block);
147   mc_chr_upload(0x6000, block, 32);
148   mc_sprite_sheet(MC_CHR(0x6000));
149   mc_display(DISP_ON | DISP_LAYER(0) | DISP_LAYER(1) | DISP_LAYER(2) | DISP_SPRITES);
150
151   print(1, 0, "CARNET   640 X 480 ENTRELACE, 80 X 30 CARACTERES", P_TITLE);
152   static const char *const intro[] = {
153     "LA HAUTE RESOLUTION SERT AUX EDITEURS ET AUX OUTILS : BEAUCOUP DE TEXTE, NET.",
154     "LES JEUX GARDENT EN GENERAL LA BASSE RESOLUTION ET SES GROS PIXELS.",
155     "",
156     "TAPEZ VOTRE TEXTE ; FLECHES, ENTREE ET RETOUR ARRIERE POUR LE MODIFIER." };
157   for (int y = 0; y < ROWS; y++) { memset(page[y], ' ', COLS); page[y][COLS] = 0; }
158   for (int i = 0; i < 4; i++) print(1, TOP + i, intro[i], i == 3 ? P_TEXT : P_DIM);
159   cy = TOP + 5;
160   status();
161   for (;;) {
162     mc_frame();
163     int e;
164     while ((e = mc_key_event()) >= 0) if (e & 0x200) type(e & 0x1FF);
165     // the cursor blinks: twice a second
166     if ((mc_frames >> 4) & 1) mc_sprite_hide(0);
167     else { mc_sprite(0, cx * 8, cy * 16, 0, 1, 1, P_CURSOR, 4); mc_sprite_zoom(0, 0x100,
0x200); }
168   }
169 }
```

Fonction	Rôle
`make_font`	fabrique la police 8 x 16 : chaque caractère de la police du SDK devient deux tuiles de 8 x 8, ses lignes doublées
`put`	écrit un caractère : ses deux tuiles, dans le plan 0 pour les lignes 0 à 14, dans le plan 1 pour les lignes 15 à 29
`text_layers`	règle les deux plans de texte et leurs tables de défilement par ligne
`make_frame`	le cadre : un plan de tuiles de 16 x 16 qui couvre tout l'écran
`key_char`	le caractère d'une touche, en AZERTY ou en QWERTY
`type`	ce que fait chaque touche : écrire, effacer, déplacer le curseur

Deux plans pour une seule couche de texte

Le plan 0 porte les lignes de texte 0 à 14, soit les lignes d'image 0 à 239 ; le plan 1 porte les lignes de texte 15 à 29. Comme chaque carte se répète tous les 256 pixels, le plan 0 se montrerait aussi en bas de l'écran, et le plan 1 en haut. La table de défilement par ligne l'empêche : dans la moitié qui n'est pas la sienne, chaque ligne du plan est décalée vers les rangées 30 et 31 de sa carte, que le programme laisse vides. Le plan 1 est en outre décalé de 16 lignes, pour que la ligne d'image 240 montre la rangée 0 de sa carte.

```
ls[y].dy = mine ? 0 : (l ? 224 : 240) + (y & 15) - y;   // elsewhere: map rows 30-31, empty
```

Des polices plus belles

La police du SDK, doublée, a l'air d'un écran de 1980. MONOTRACKER et SOLFEGE rendent leur police à partir d'une vraie police vectorielle (DejaVu Sans Mono) avec un petit script Python, `make_font.py` ou `make_assets.py`, et gardent quatre niveaux de gris : le programme règle ses palettes en mélangeant la couleur du texte avec celle du fond, ce qui lisse les bords. SOLFEGE rend de la même façon les clés et les notes d'une police musicale.

Les sprites en haute résolution

Les sprites se placent en pixels de l'image complète : en 640 x 480, un sprite de 16 x 16 paraît deux fois plus petit qu'en 320 x 240. `mc_sprite_zoom` le grandit sans dessin supplémentaire ; c'est ce que fait CARNET pour son curseur, une case de 8 x 8 doublée en hauteur. Le budget par ligne ne change pas : 64 sprites et 1280 pixels de sprites.

Pour aller plus loin

1. Ajoutez une ligne de règle (les numéros de colonnes) sous la barre de titre.
2. Gardez le texte dans la mémoire sauvegardée (NVRAM), pour le retrouver au prochain lancement.
3. Changez la police : rendez-en une avec le `make_font.py` de MONOTRACKER.
4. Passez CARNET en 640 x 240, sans entrelacement : une seule carte de 128 x 32 tuiles de 8 x 8 couvre alors l'écran, et les caractères redeviennent des cases de 8 x 8.



La bibliothèque easy

Toutes les fonctions, d'un coup d'oeil, avec la traduction de leur nom.

Fonction	Traduction	Description
`init()`	initialiser	prépare la console : à appeler une fois, au début
`wait_frame()`	attendre l'image	attend l'image suivante (1/60 de seconde), lit les manettes, fait avancer les mélodies
`frame_count`	nombre d'images	variable : images depuis le démarrage
`random(n)`	au hasard	nombre au hasard de 0 à n - 1
`background(r, g, b)`	fond	couleur du fond (rouge, vert, bleu de 0 à 255)
`ink(n, r, g, b)`	encre	couleur de l'encre de texte n (0 à 7)
`load_palette(n, pal)`	charger une palette	range les couleurs d'une image en palette n (0 à 7), par exemple `load_palette(3, hero_pal)`
`cls()`	effacer l'écran	efface tout le texte (de *CLear Screen*)
`print(col, row, text)`	imprimer	écrit en blanc, colonne *col*, ligne *row*
`print_ink(col, row, text, ink)`	imprimer en couleur	écrit avec l'encre choisie
`print_num(col, row, n, digits, ink)`	imprimer un nombre	écrit un nombre sur *digits* chiffres, complété par des zéros
`held(player, button)`	tenu	vrai si le bouton du joueur (1 à 4) est enfoncé
`pressed(player, button)`	appuyé	vrai si le bouton du joueur (1 à 4) vient d'être enfoncé
`sprite(n, x, y, picture)`	sprite	affiche le sprite n (0 à 63) avec le dessin *picture*
`hide_sprite(n)`	cacher	cache le sprite n
`flip_sprite(n, h, v)`	retourner	retourne le sprite n
`scale_sprite(n, percent)`	mettre à l'échelle	agrandit ou réduit le sprite n
`sprite_palette(n, palette)`	palette du sprite	change la palette du sprite n (-1 : celle de sa planche)

Fonction	Traduction	Description
`collide(a, b)`	se heurter	vrai si les sprites a et b se touchent
`load_map(map)`	charger le décor	remplace le décor affiché par `init()`
`scroll_map(x, y)`	faire défiler	fait défiler le décor
`tone(channel, freq, volume)`	note	son carré sur un canal
`noise(channel, volume)`	bruit	bruit sur un canal
`quiet(channel)`	silence	fait taire un canal
`play(channel, tune, frames, volume)`	jouer	joue une mélodie en boucle
`stop(channel)`	arrêter	arrête la mélodie du canal
`play_sample(voice, sound, volume)`	jouer un échantillon	joue un son enregistré

Boutons : `PAD_UP` (haut), `PAD_DOWN` (bas), `PAD_LEFT` (gauche), `PAD_RIGHT` (droite), `PAD_B1` à `PAD_B6`, `PAD_START`, `PAD_COIN` (pièce). Notes pour `tone()` : `NOTE_C2` à `NOTE_B7` (`NOTE_A4` = 440 Hz, `NOTE_CS4` = DO dièse, `NOTE_Db4` = RÉ bémol). Encres par défaut : 0 blanc, 1 jaune, 2 cyan, 3 vert, 4 rouge, 5 violet, 6 orange, 7 gris.

LE FICHIER PROJECT.TXT

Réglage	Contenu	Noms créés pour le programme
`name`	nom du fichier produit (`build/name.mnc`)	
`title`	titre inscrit dans la ROM	
`sources`	fichiers C du programme	
`sprites`	planches de sprites (PNG de 16, 32 ou 64 pixels de large)	`x` : numéro de son premier dessin
`maps`	décors (PNG, tuiles de 16 x 16)	`x` : le décor, pour `load_map`
`sounds`	sons (WAV)	`x` : le son, pour `play_sample`
`origin`	origine inscrite dans la ROM (facultatif)	
`rotation`	orientation de l'écran au démarrage : 0 à 3 quarts de tour (facultatif)	
`dip`	DIP switches par défaut, par exemple `0x01` (facultatif)	

`x` est le nom du fichier sans son extension : `sheet.png` donne `sheet`. Les planches puis les décors reçoivent chacun une palette de couleurs, dans l'ordre de `project.txt` (0, 1, 2...). Les données brutes existent aussi sous les noms `x_chr`, `x_pal`, `x_map`, `x_data`... (utiles avec `load_palette` ou le SDK **mc**).

Carte mémoire et registres

Pour programmer la console directement (monocore.h).

Adresse	Taille	Contenu
`0x0000_0000`	32 Mo max	ROM : programme, dessins, sons (lecture seule)
`0x1000_0000`	128 Ko	mémoire de travail : variables, pile
`0x1800_0000`	32 Ko	mémoire sauvegardée (scores, réglages)
`0x2000_0000`	4 Ko	registres vidéo
`0x2001_0000`	8 Ko	palette : 4096 couleurs RGB555
`0x2002_0000`	8 Ko	table des 512 sprites, 16 octets chacun
`0x2004_0000`	64 Ko	cartes des 4 plans, 16 Ko chacune
`0x2008_0000`	8 Ko	défilement par ligne des 4 plans
`0x200C_0000`	128 Ko	CHR RAM : dessins modifiables
`0x3000_0000`	4 Ko	registres son
`0x4000_0000`	4 Ko	manettes, interruptions, timers, DMA

VIDÉO

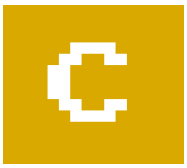
Registre	Rôle
`DISP_CTRL`	bit 0 affichage, bits 1 à 4 plans 0 à 3, bit 5 sprites, bits 8-9 rotation
`SCREEN_W / SCREEN_H`	taille de l'image (224-384 x 192-288)
`BG_COLOR`	numéro de la couleur de fond dans la palette
`LINE_CMP`	ligne de l'interruption LINE
`VCOUNT / FRAME`	ligne en cours, numéro d'image (lecture)
`SPR_CHR / SPR_COUNT`	adresse des dessins de sprites, nombre d'entrées parcourues
`LAYER_CTRL(n)`	plan n : tuiles 16 (bit 0), 8 bpp (bit 1), largeur et hauteur de carte, défilement par ligne (bit 6), priorité (bits 8-9)
`LAYER_CHR(n)`	adresse des dessins du plan n
`LAYER_SCROLLX/Y(n)`	défilement du plan n

SON

Registre	Rôle
`PCM[v].start / length / loop / step`	voix d'échantillons v (0 à 7) : adresse, longueur, boucle, pas (0x10000 = 48 kHz)
`PCM[v].vol_l / vol_r / format`	volumes, format (0 : 8 bits, 1 : 16 bits, 2 : ADPCM)
`PCM[v].ctrl`	1 lance, 0 arrête
`PSG[c].freq`	fréquence du canal c en 1/16 de hertz
`PSG[c].wave / duty`	forme (carré, triangle, dent de scie, bruit), rapport cyclique
`PSG[c].vol_l / vol_r`	volumes
`MASTER_VOL`	volume général

ENTRÉES, INTERRUPTIONS, TIMERS, DMA

Registre	Rôle
`IN_P1 / IN_P2 / IN_SYS / DIPSW`	manettes, boutons système, DIP switches
`HW_VERSION`	version du matériel : 2 et plus, les entrées ci-dessous existent ; 3 la souris, 4 l'entrée MIDI, 5 la haute résolution et le mode entrelacé
`IN_P3 / IN_P4`	joueurs 3 et 4, même format que `IN_P1`
`ANALOG_P(n)`	manette n (0 à 3) : sticks gauche et droit, X et Y de −127 à +127
`PADDLE_P(n)`	paddle de 0 à 255 (bits 0-7), position du spinner (bits 16-31)
`KEY_STATE(k)`	clavier : 8 mots de 32 bits, un bit par touche enfoncée
`KEY_EVENT`	plus ancien événement clavier (bit 31 valide, bit 9 appui) ; écrire pour le retirer
`IRQ_STATUS / IRQ_ENABLE`	interruptions en attente (écrire 1 pour acquitter) et autorisées
`T0_RELOAD / T0_CTRL`	timer 0 : période en cycles, marche
`CYCLE_LO / CYCLE_HI`	compteur de cycles du processeur (50 MHz)
`DMA_SRC / DST / LEN / CTRL`	copie (CTRL = 1) ou remplissage (CTRL = 3)
`DEBUG_OUT`	un caractère vers la console de debug du simulateur



Limites et dépannage

Ce que la console ne peut pas faire, et que faire quand rien ne marche.

Limite	Valeur
Sprites par ligne	64, et 1280 pixels de sprites
Couleurs	15 par planche ou par décor (4 bits par pixel), 255 en mode 8 bits
Décor	1024 x 1024 pixels (64 x 64 tuiles de 16)
Mémoire de travail	128 Ko pour les variables et la pile
ROM	32 Mo
Son	8 voix d'échantillons + 8 canaux de notes

MESSAGES ET PROBLÈMES FRÉQUENTS

Symptôme	Cause probable et remède
`error: expected ';'`	il manque un point-virgule à la fin de la ligne indiquée (ou de la précédente)
`use of undeclared identifier 'sheet'`	l'image n'est pas nommée dans `project.txt`, ou porte un autre nom
`ERROR: … 8 at most`	plus de 8 planches et décors : regroupez des dessins dans une même planche
`ERROR: tool not found: clang`	LLVM n'est pas installé ou pas dans le chemin (chapitre 2)
`ERROR: no project.txt file`	la commande n'est pas lancée dans le dossier du programme
écran noir	`init()` oublié, ou le programme n'appelle jamais `wait_frame()`
sprite invisible	numéro de dessin au-delà du dernier, sprite hors de l'écran, ou plus de 64 sprites sur la ligne
couleurs bizarres	plus de 15 couleurs dans l'image : réduisez la palette
le jeu ralentit	trop de calculs par image : regardez la charge dans le moniteur
rien sur le MiSTer	barrette SDRAM absente, ou essayer l'option « Lecture SDRAM : Retardée »

Le SDK mc

Toutes les fonctions du SDK bas niveau, d'un coup d'oeil (mc.h).

Fonction	Description
`mc_init()`	prépare la console : à appeler une fois, au début
`mc_frame()`	attend l'image suivante, envoie les sprites, lit les manettes, fait avancer les mélodies
`mc_frames`	variable : images depuis le démarrage
`mc_cycles() / mc_micros()`	temps écoulé, en cycles du processeur ou en microsecondes
`mc_on_vblank(f)`	appelle *f* au début de chaque pause verticale
`mc_on_timer(t, hz, f)`	appelle *f* *hz* fois par seconde (timer 0 ou 1)
`mc_raster_add(ligne, f)`	effet raster : *f* s'applique à partir de cette ligne (64 au plus)
`mc_raster_line / mc_raster_clear()`	ligne de l'effet en cours ; efface tous les effets
`mc_video_mode(w, h)`	taille de l'écran, de 224 à 384 x 192 à 288 ; en version 5, jusqu'à 640 de large, et 384 à 512 lignes en entrelacé (chapitre 13)
`mc_screen_w() / mc_screen_h()`	la taille de l'image (en entrelacé, la hauteur complète)
`mc_interlaced() / mc_field()`	le mode entrelacé est-il actif ; la parité de la trame en cours
`mc_display(flags)`	allume l'affichage, les plans et les sprites (`DISP_ON | DISP_LAYER(n) | DISP_SPRITES`)
`mc_color(i, rgb) / MC_RGB(r, g, b)`	change la couleur *i* (0 à 4095)
`mc_palette(p, couleurs, n)`	charge *n* couleurs à partir de la palette *p*
`mc_background(i)`	choisit la couleur de fond
`mc_layer(n, dessins, flags, w, h, prio)`	règle le plan *n* : tuiles 8 ou 16, 4 ou 8 bits, taille de carte, priorité
`mc_layer_scroll(n, x, y)`	fait défiler le plan *n*

Fonction	Description
`mc_layer_linescroll(n, on) / mc_linescroll(n)`	défilement ligne par ligne et sa table
`mc_tile / mc_tilemap_fill / mc_tilemap_load`	écrit une case, remplit ou charge la carte d'un plan
`MC_TILE(tuile, palette, flags)`	une case de carte, avec `T_HFLIP`, `T_VFLIP`, `T_PRIO`
`mc_chr_upload(décalage, source, octets)`	copie des dessins dans la CHR RAM
`MC_CHR(décalage)`	adresse de dessins dans la CHR RAM, pour un plan ou les sprites
`mc_sprite_sheet(dessins)`	choisit la planche des sprites (ROM ou CHR RAM)
`mc_sprite(n, x, y, case, w, h, pal, prio)`	affiche le sprite *n* (0 à 511), taille en cases de 8 pixels
`mc_sprite_zoom / mc_sprite_flip / mc_sprite_8bpp`	agrandit, retourne, passe en 256 couleurs
`mc_sprite_hide / mc_sprites_hide_all`	cache un sprite, ou tous
`mc_sprites_used(n)`	nombre d'entrées parcourues par la console
`mc_text_layer(n, prio) / mc_text / mc_text_clear`	texte avec la police du SDK sur le plan *n*
`mc_psg(c, forme, hz, rapport, vol_g, vol_d)`	note sur le canal *c* (`PSG_SQUARE`, `PSG_TRIANGLE`, `PSG_SAW`, `PSG_NOISE`)
`mc_psg_volume / mc_psg_freq16`	volume, fréquence en 1/16 de hertz
`mc_pcm_play(v, données, n, format, hz, boucle, vol_g, vol_d)`	joue un échantillon sur la voix *v*
`mc_pcm_stop / mc_pcm_pos / mc_master_volume`	arrête une voix, sa position, volume général
`mc_tune(c, notes, images, volume) / mc_tune_stop`	mélodie en boucle, même écriture que `play` d'easy
`mc_pad(j) / mc_pad_pressed(j) / mc_pad_released(j)`	boutons tenus, enfoncés ou relâchés à cette image (joueur 1 à 4)
`mc_sys() / mc_dip()`	boutons système, DIP switches
`mc_hw_version()`	version du matériel : 2 et plus pour les joueurs 3 et 4, l'analogique et le clavier
`mc_stick_x(j, s) / mc_stick_y(j, s)`	stick *s* (0 gauche, 1 droit) du joueur *j*, de −127 à +127
`mc_paddle(j) / mc_spinner(j) / mc_spinner_delta(j)`	paddle (0 à 255), position du spinner, son déplacement depuis l'image précédente
`mc_key(k) / mc_key_pressed(k) / mc_key_released(k)`	touche `KEY_xxx` tenue, enfoncée ou relâchée à cette image
`mc_key_event()`	événement clavier suivant (0x200 appui, 0x100 touche étendue, + code), −1 s'il n'y en a plus

Fonction	Description
`malloc / calloc / realloc / free`	mémoire du tas ; `mc_heap_free()` donne ce qui reste
`MC_NVRAM`	32 Ko de mémoire sauvegardée
`mc_copy / mc_fill`	copie ou remplissage par DMA
`fix16, FIX(x), fix_mul, fix_div`	nombres à virgule fixe (16 bits après la virgule)
`mc_sin / mc_cos / mc_atan2 / mc_isqrt`	trigonométrie (1024 pas par tour) et racine carrée
`mc_rand() / mc_srand(n)`	nombres au hasard
`mc_debug / printf / puts`	écriture sur la console de debug du simulateur

LE FICHIER PROJECT.TXT

Réglage	Contenu	Noms créés pour le programme
`library = mc`	choisit le SDK mc (par défaut : easy)	
`data`	fichiers bruts, copiés tels quels dans la ROM	`x_file` `x_size`
`maps8`	décors en 256 couleurs (8 bits par pixel, tuiles de 16 x 16)	`x_chr` `x_pal` `x_map` …
`cflags`	options de compilation en plus pour les fichiers du programme (chemins d'inclusion, avertissements)	

MONOCORE-1 · Manuel du programmeur · édition 3, octobre 2026.

Titres en Press Start 2P (Cody Boisclair), texte en Archivo (Omnibus-Type), listings en IBM Plex Mono : polices sous licence SIL Open Font License.

Les exemples se trouvent dans `manual/examples`. Ce manuel se régénère avec `python3 manual/tools/manual.py`.